Lecture 14. Catalytic Computation & Time-Space Complexity

Spring 2026, *Theory of Computation*, IIIS, Tsinghua

Overview

Reversible Program

Catalytic Space

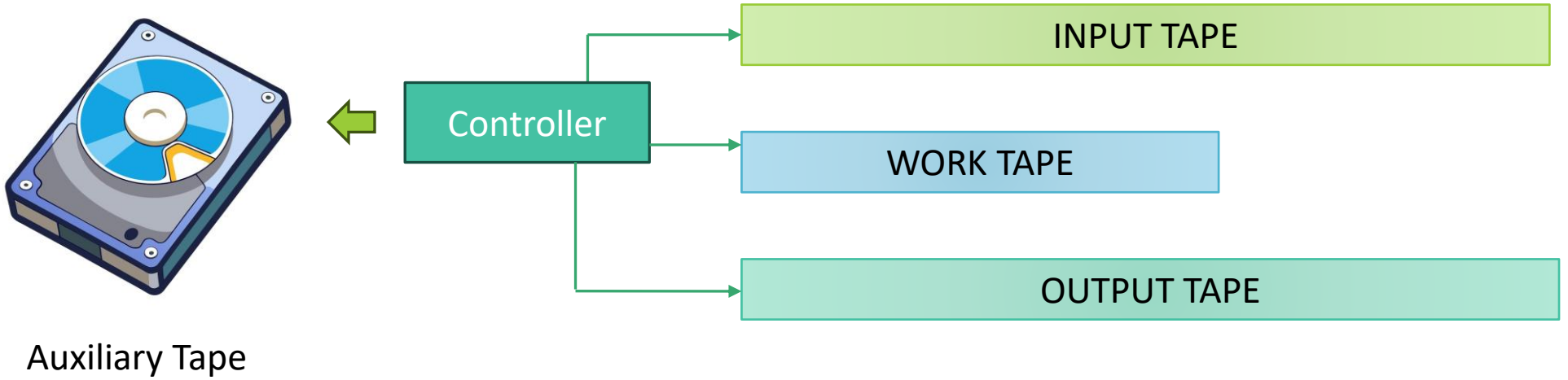
Tree Evaluation

Simulating Time with Space

Catalytic space

In catalytic computing, a TM is given additional memory already full of arbitrary data whose contents must be preserved by the computation

- the catalytic TM must have returned the memory to its initial contents



Catalytic Classes

CSPACE[s(n),c(n)] is the set of all languages that can be decided by a catalytic TM using O(s(n)) space of the work tape and O(c(n)) space of the auxiliary tape, for an input of length n

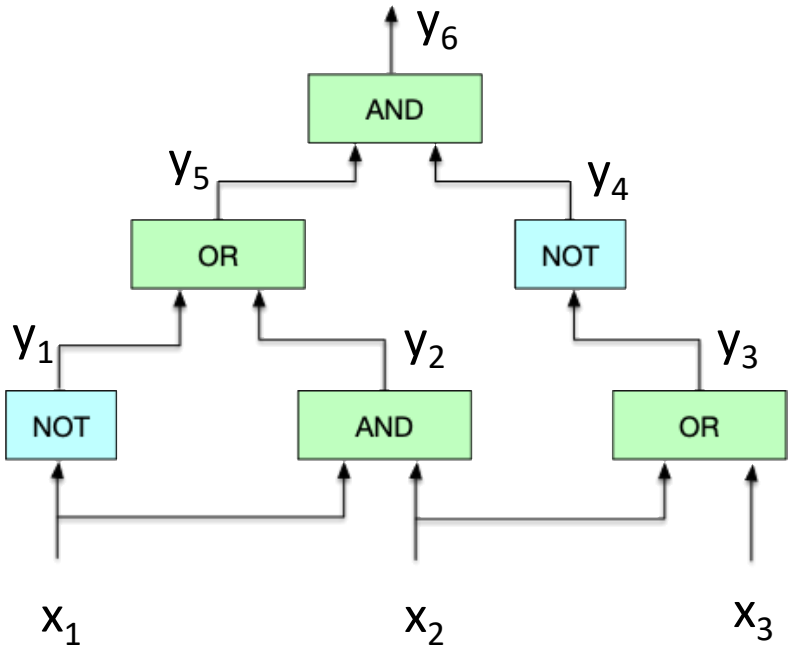
Usually CSPACE[s(n)]=CSPACE[s(n), $2^{O(s(n))}$], and CL=CSPACE[log n]

- CL: logspace work tape, polynomial length of auxiliary tape

Straight-line program

Straight-line programs: contain no if…then, goto, loop, etc

- equivalent to circuits



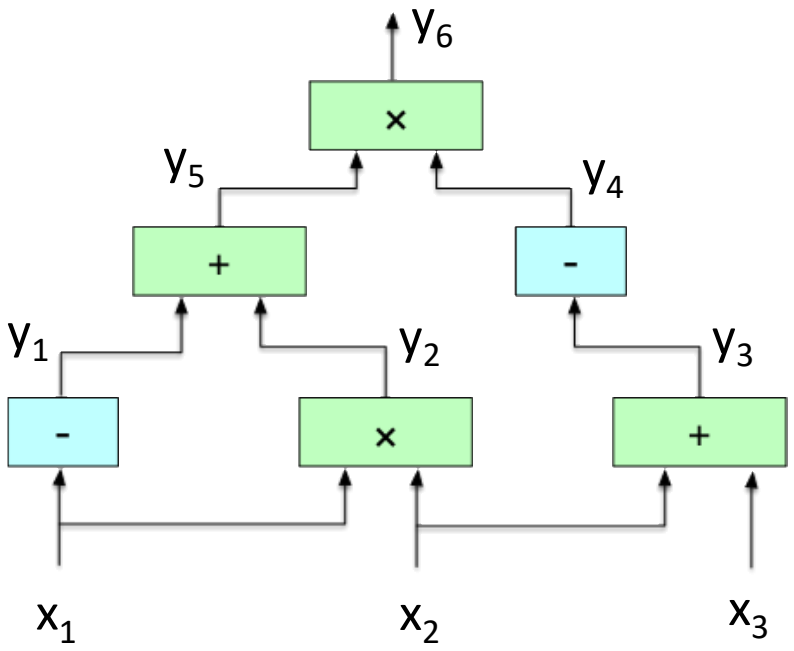


$y_1 \longleftarrow \sim x_1$
$y_2 \longleftarrow x_1 \wedge x_2$
$y_3 \longleftarrow x_2 \vee x_3$
$y_5 \longleftarrow y_1 \vee y_2$
$y_4 \longleftarrow \sim y_3$
$y_6 \longleftarrow y_5 \wedge y_4$

Straight-line program

Straight-line programs: contain no if…then, goto, loop, etc

- also works for algebraic circuit:





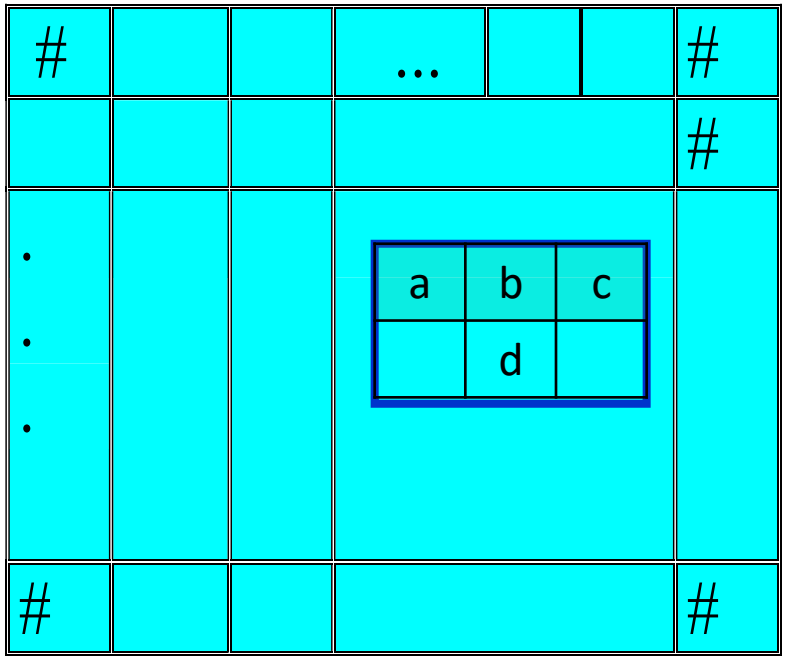
$y_1 \longleftarrow -x_1$
$y_2 \longleftarrow x_1 * x_2$
$y_3 \longleftarrow x_2 + x_3$
$y_5 \longleftarrow y_1 + y_2$
$y_4 \longleftarrow -y_3$
$y_6 \longleftarrow y_5 * y_4$

Straight-line program

Also have uniformity and non-uniformity

a TM in $t(n)$ time $\Leftrightarrow$ straight-line program of length $O(t(n)^2)$

- generate $t(n) \times t(n)$ table of computation history



Reversible program

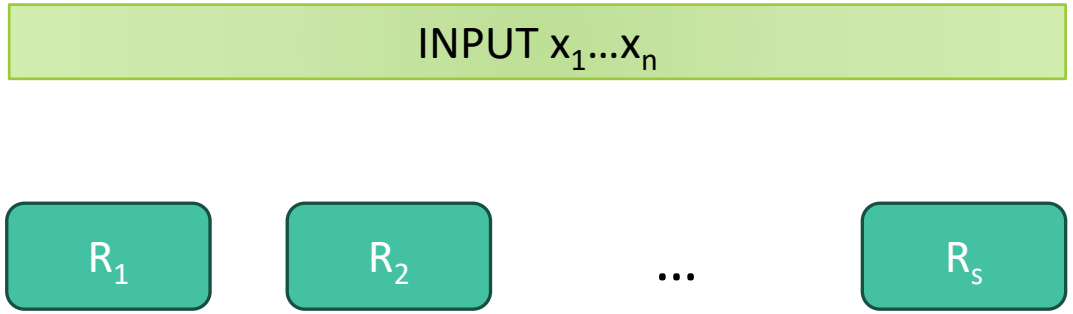
Straight-line program P on registers $\{R_1, R_2, \ldots, R_s\}$ over ring $\mathcal{R}$

- $\mathcal{R}$ can be a ring or a field

Standard set of instructions:

$$R_i \longleftarrow R_i \pm u \times v$$

where u,v are either constant from $\mathcal{R}$, inputs x_j or register R_k for $k \neq i$



straight-line program

Reversible program

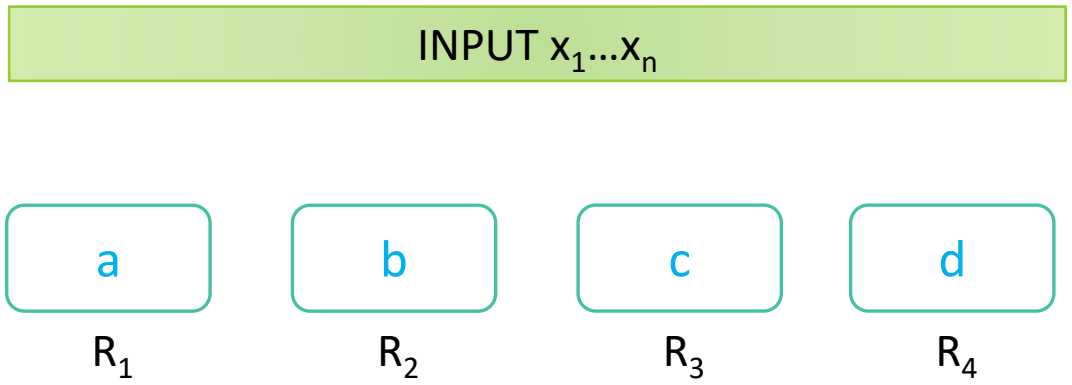
Straight-line program P on registers $\{R_1, R_2, \ldots, R_s\}$ over ring $\mathcal{R}$

- $\mathcal{R}$ can be a ring or a field

Standard set of instructions:

$$R_i \longleftarrow R_i \pm u \times v$$

where u,v are either constant from $\mathcal{R}$, inputs x_j or register R_k for $k \neq i$



straight-line program

Instruction $\mathbf{I}$:

$$R_1 \longleftarrow R_1 + R_2 \times R_3$$

Reversible program

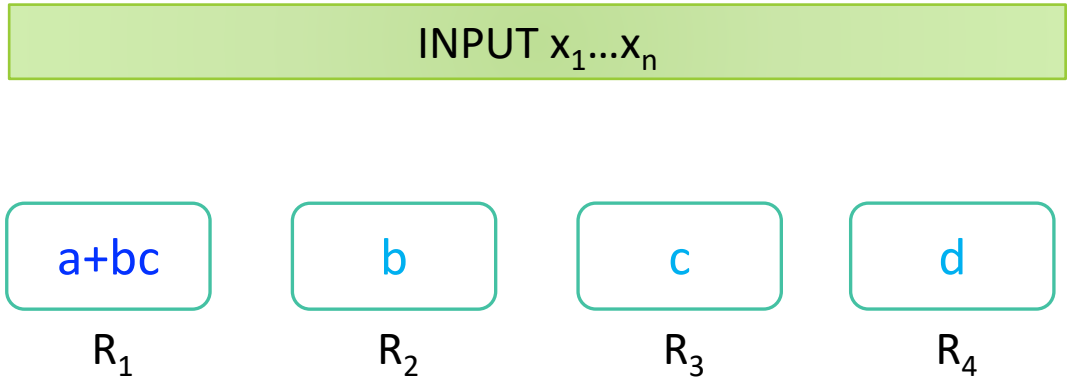
Straight-line program P on registers $\{R_1, R_2, \ldots, R_s\}$ over ring $\mathcal{R}$

- $\mathcal{R}$ can be a ring or a field

Standard set of instructions:

$$R_i \longleftarrow R_i \pm u \times v$$

where u,v are either constant from $\mathcal{R}$, inputs x_j or register R_k for $k \neq i$



straight-line program

Instruction $\mathbf{I}$:

$$R_1 \longleftarrow R_1 + R_2 \times R_3$$

its inverse $\mathbf{I}^{-1}$:

$$R_1 \longleftarrow R_1 - R_2 \times R_3$$

Reversible program

Straight-line program P on registers $\{R_1,R_2,\ldots,R_s\}$ over ring $\mathcal{R}$

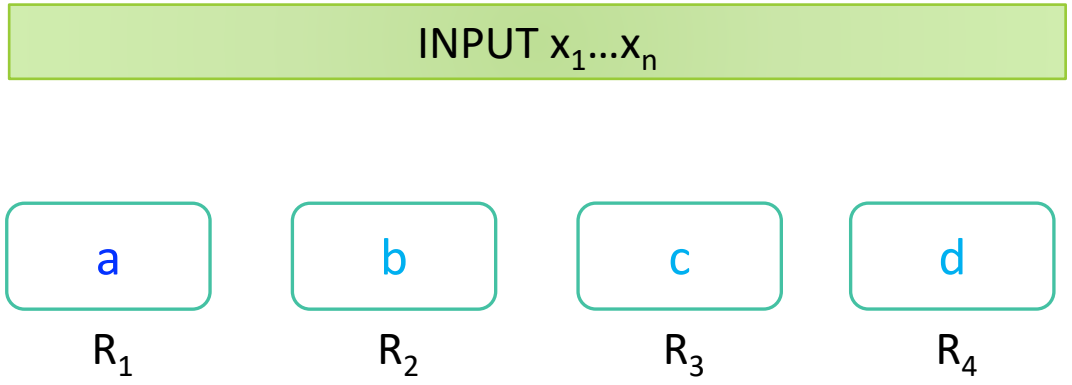
- $\mathcal{R}$ can be a ring or a field

Standard set of instructions:

$$R_i \longleftarrow R_i \pm u \times v$$

where u,v are either constant from $\mathcal{R}$, inputs x_j or register R_k for $k \neq i$

So P is reversible with inverse P^{-1}



straight-line program

Instruction $\mathbf{I}$:

$$R_1 \longleftarrow R_1 + R_2 \times R_3$$

its inverse $\mathbf{I}^{-1}$:

$$R_1 \longleftarrow R_1 - R_2 \times R_3$$

Reversible program

Straight-line program P on registers $\{R_1,R_2,\ldots,R_s\}$ over ring $\mathcal{R}$

- $\mathcal{R}$ can be a ring or a field

Standard set of instructions:

$$R_i \longleftarrow R_i \pm u \times v$$

where u,v are either constant from $\mathcal{R}$, inputs x_j or register R_k for $k \neq i$

So P is reversible with inverse P^{-1}

program P:

$R_1 \longleftarrow R_1 + R_2 \cdot R_3$
$R_3 \longleftarrow R_3 - b$
$R_2 \longleftarrow R_2 - c \cdot R_1$
$R_4 \longleftarrow R_4 + x_2 \cdot R_1$



inverse P^{-1}:

$R_4 \longleftarrow R_4 - x_2 \cdot R_1$
$R_2 \longleftarrow R_2 + c \cdot R_1$
$R_3 \longleftarrow R_3 + b$
$R_1 \longleftarrow R_1 - R_2 \cdot R_3$

Other possible forms

Straight-line program P on registers $\{R_1,R_2,\ldots,R_s\}$ over ring $\mathcal{R}$

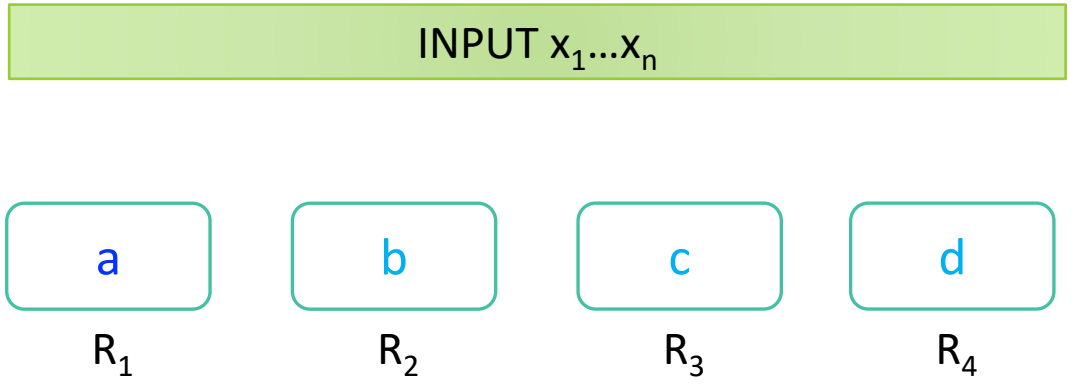
- $\mathcal{R}$ can be a ring or a field

in fact, we can update register R_i as:

$$R_i \longleftarrow R_i + p(R_1,\ldots,R_{i-1},R_{i+1},\ldots,R_s) \qquad \textit{for a polynomial } p(\cdot)$$

or even $R_i \longleftarrow a \cdot R_i$ for $a \in \mathcal{R}$ if $\mathcal{R}$ is a field

- but no $R_1 \longleftarrow R_2 \cdot R_3$ or $R_1 \longleftarrow \min\{R_1,R_3\}$

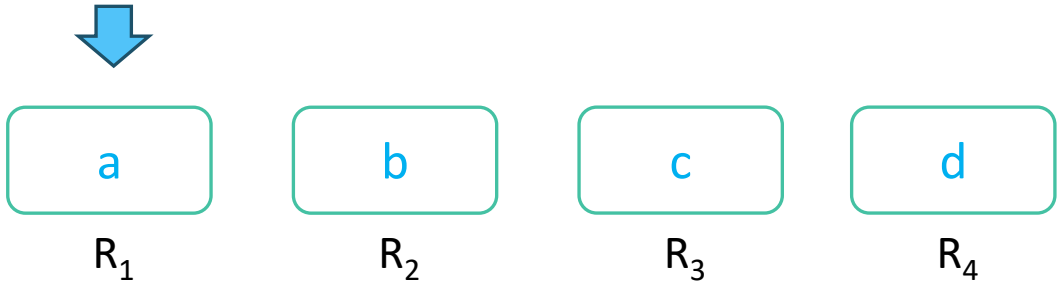


straight-line program

Transparent computing

a function $f(x_1,\ldots,x_n)$ can be *computed transparently* into a register R_i if there is a reversible program P such that when executed on registers $R_1,R_2,\ldots,R_s$ with initial values $\tau_1,\tau_2,\ldots,\tau_s$ we have:

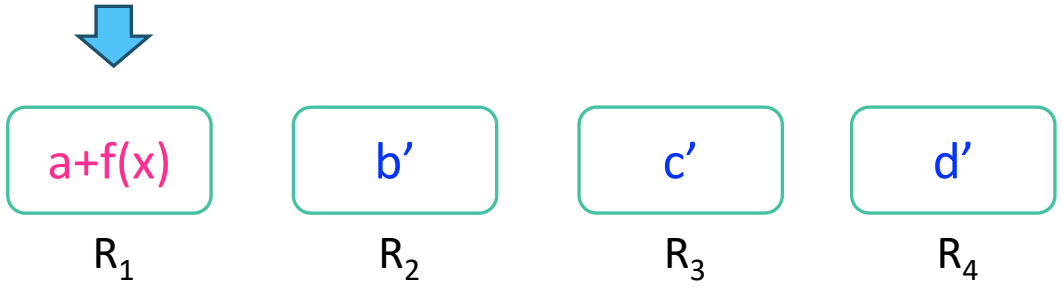
- value $\tau_i + f(x_1,\ldots,x_n)$ in register R_i
- the other registers may contain arbitrary values at the end of the computation



Transparent computing

a function $f(x_1,\ldots,x_n)$ can be *computed transparently* into a register R_i if there is a reversible program P such that when executed on registers $R_1,R_2,\ldots,R_s$ with initial values $\tau_1,\tau_2,\ldots,\tau_s$ we have:

- value $\tau_i + f(x_1,\ldots,x_n)$ in register R_i
- the other registers may contain arbitrary values at the end of the computation

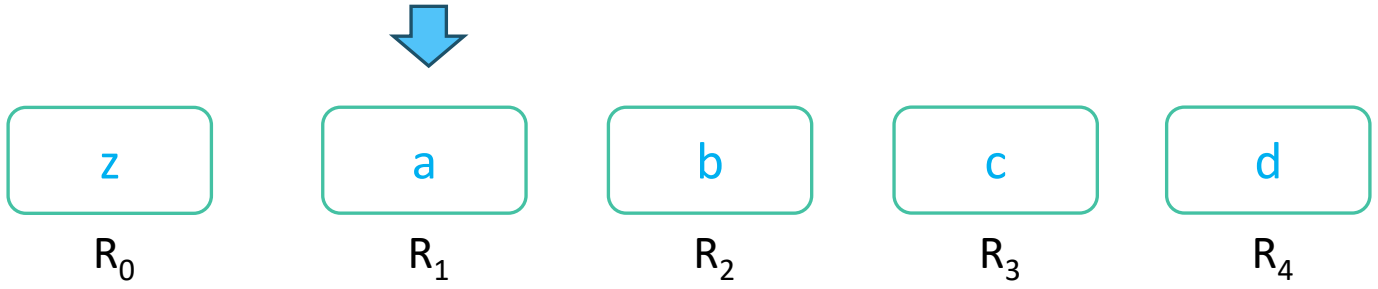


Transparent computing

a function $f(x_1,\ldots,x_n)$ can be *computed transparently* into a register R_i if there is a reversible program P such that when executed on registers $R_1,R_2,\ldots,R_s$ with initial values $\tau_1,\tau_2,\ldots,\tau_s$ we have:

- value $\tau_i + f(x_1,\ldots,x_n)$ in register R_i
- the other registers may contain arbitrary values at the end of the computation

Suppose P transparently computes f into R_1



add another register R_0

Transparent computing

Suppose P transparently computes f into R_1

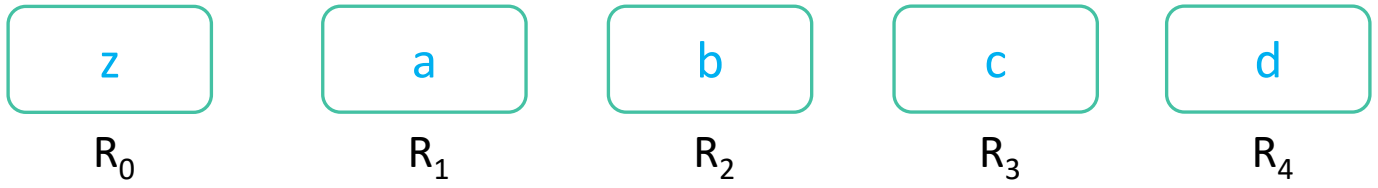
Run P':

$R_0 \longleftarrow R_0 - R_1$

P

$R_0 \longleftarrow R_0 + R_1$

P^{-1}



add another register R_0

Transparent computing

Suppose P transparently computes f into R_1

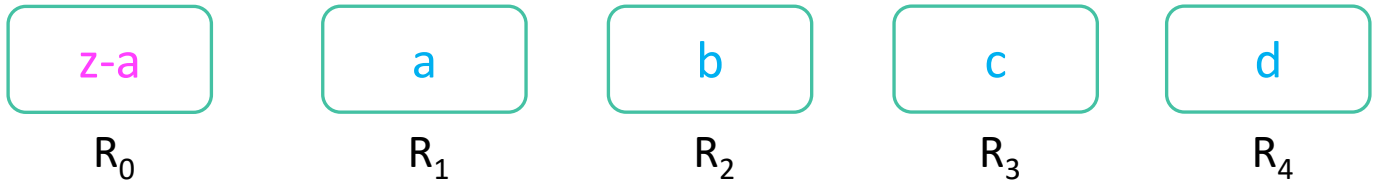
Run P':

$R_0 \longleftarrow R_0 - R_1$

P

$R_0 \longleftarrow R_0 + R_1$

P^{-1}



add another register R_0

Transparent computing

Suppose P transparently computes f into R_1

Run P':

$R_0 \longleftarrow R_0 - R_1$

P

$R_0 \longleftarrow R_0 + R_1$

P^{-1}

z-a	a+f	b'	c'	d'
R_0	R_1	R_2	R_3	R_4

add another register R_0

Transparent computing

Suppose P transparently computes f into R_1

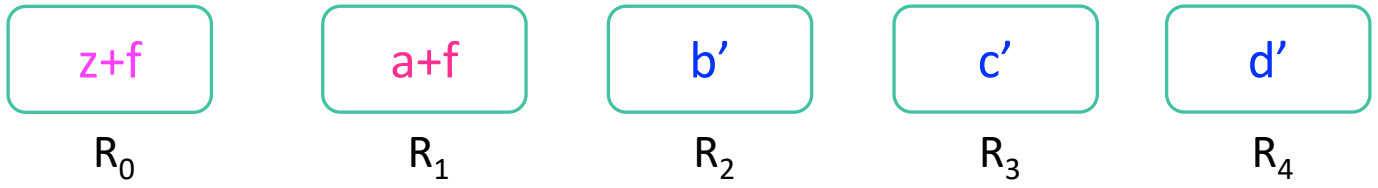
Run P':

$R_0 \leftarrow R_0 - R_1$

P

$R_0 \leftarrow R_0 + R_1$

P^{-1}



add another register R_0

Transparent computing

Suppose P transparently computes f into R_1

Run P':

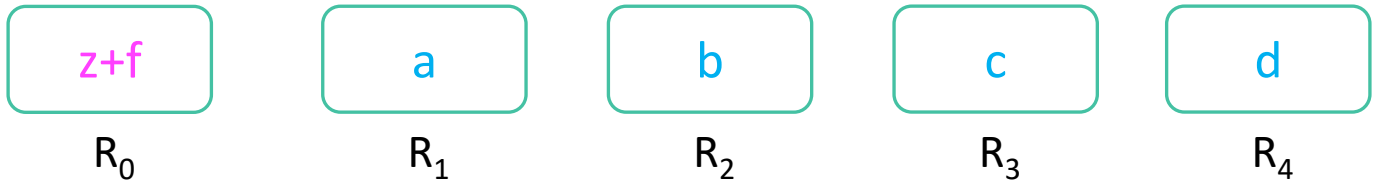
$R_0 \leftarrow R_0 - R_1$

P

$R_0 \leftarrow R_0 + R_1$

P^{-1}

we say P' cleanly compute f into R_0



add another register R_0

Clean computing

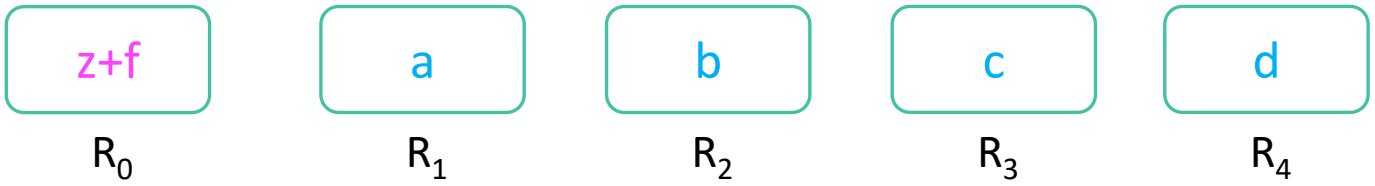
a function $f(x_1,\ldots,x_n)$ can be *computed transparently* into a register R_i if there is a reversible program P such that when executed on registers $R_1,R_2,\ldots,R_s$ with initial values $\tau_1,\tau_2,\ldots,\tau_s$ we have:

- value $\tau_i + f(x_1,\ldots,x_n)$ in register R_i
- the other registers may contain arbitrary values at the end of the computation

a function $f(x_1,\ldots,x_n)$ can be *computed cleanly* into a register R_i if there is a program P such that when executed on registers $R_1,R_2,\ldots,R_s$ with initial values $\tau_1,\tau_2,\ldots,\tau_s$ we have:

- value $\tau_i + f(x_1,\ldots,x_n)$ in register R_i
- value τ_k in register R_k for $k \neq i$

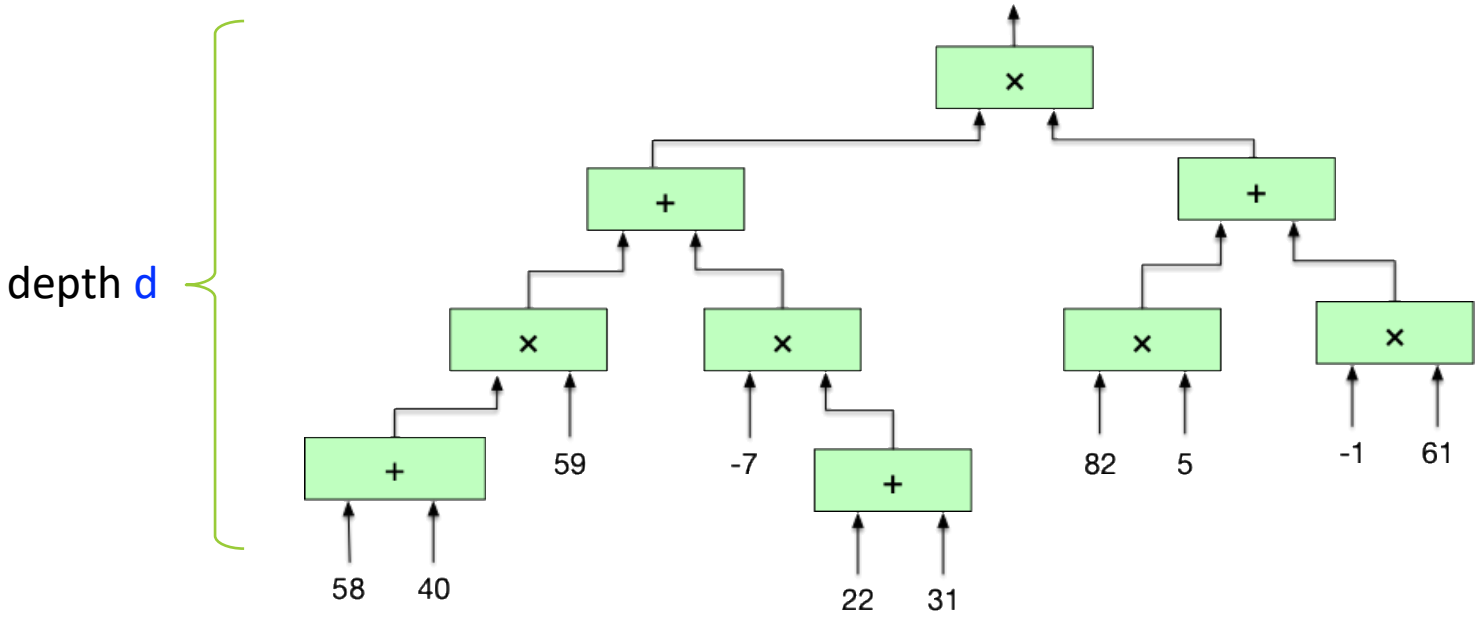
Transparent computing can easily change to clean computing with one more register



Algebraic formula over ring $\mathcal{R}$

a formula is a circuit whose graph is a tree

e.g. ((58 + 40) × 59 - 7×(22+31)) × (82 × 5 - 61)

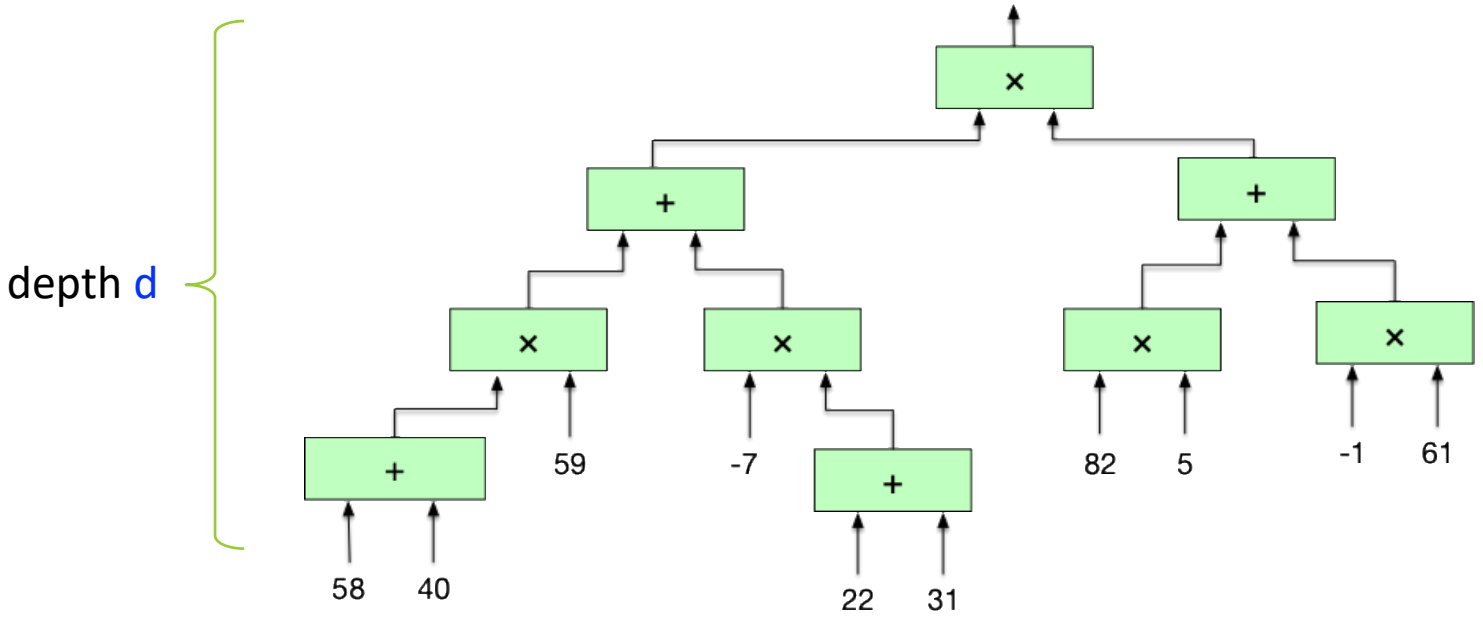


change to straight-line program with O(d) registers:

$R_1 \leftarrow 58+40$
$R_2 \leftarrow R_1 \times 59$
$R_1 \leftarrow 22+31$
$R_3 \leftarrow (-7) \times R_1$
$R_4 \leftarrow R_2 + R_3$
$R_1 \leftarrow 82 \times 5$
$R_2 \leftarrow (-1) \times 61$
$R_3 \leftarrow R_1 + R_2$
$R_5 \leftarrow R_4 \times R_3$

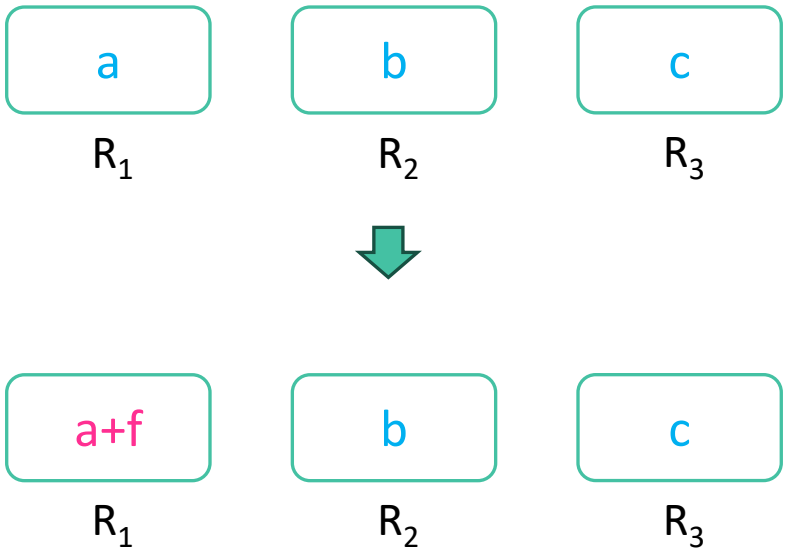
Algebraic formula over ring $\mathcal{R}$

In fact, any algebraic formula f can be computed **cleanly** with 3 registers *[Ben-Or & Cleve 1992]*



Algebraic formula over ring $\mathcal{R}$

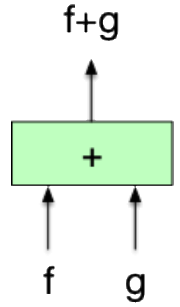
In fact, any algebraic formula f can be computed **cleanly** with 3 registers *[Ben-Or & Cleve 1992]*



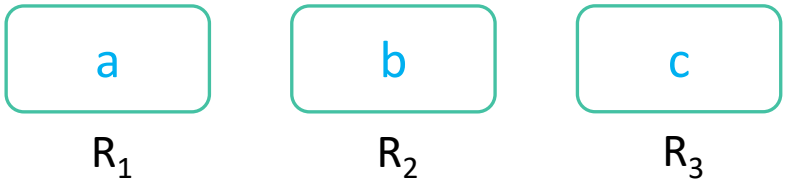
Algebraic formula over ring $\mathcal{R}$

In fact, any algebraic formula f can be computed **cleanly** with 3 registers

- trivially if f is an input or a constant
- Suppose it holds for f and g, consider f+g



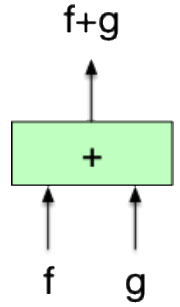
Compute f on R_1
Compute g on R_1



Algebraic formula over ring $\mathcal{R}$

In fact, any algebraic formula f can be computed **cleanly** with 3 registers

- trivially if f is an input or a constant
- Suppose it holds for f and g, consider f+g



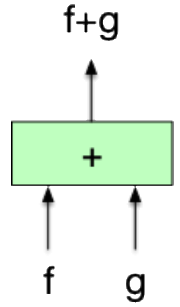
Compute f on R_1
Compute g on R_1

a+f	b	c
R_1	R_2	R_3

Algebraic formula over ring $\mathcal{R}$

In fact, any algebraic formula f can be computed **cleanly** with 3 registers

- trivially if f is an input or a constant
- Suppose it holds for f and g, consider f+g



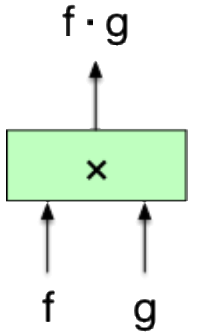
Compute f on R_1
Compute g on R_1

a+f+g	b	c
R_1	R_2	R_3

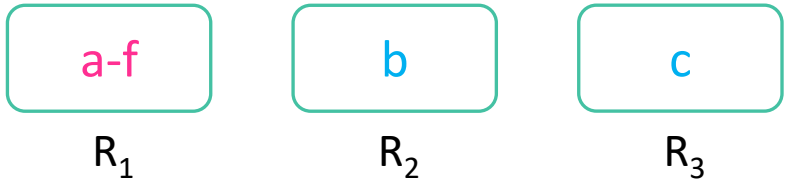
Algebraic formula over ring $\mathcal{R}$

In fact, any algebraic formula f can be computed **cleanly** with 3 registers

- trivially if f is an input or a constant
- Suppose it holds for f and g, what about f·g



If P cleanly compute f in R_1, then P^{-1} cleanly compute -f in R_1,



Algebraic formula over ring $\mathcal{R}$

In fact, any algebraic formula f can be computed **cleanly** with 3 registers

- trivially if f is an input or a constant
- Suppose it holds for f and g, what about f·g

Compute f on R_2
$R_1 \longleftarrow R_1 - R_2 \cdot R_3$
Compute g on R_3
$R_1 \longleftarrow R_1 + R_2 \cdot R_3$
Compute -f on R_2
$R_1 \longleftarrow R_1 - R_2 \cdot R_3$
Compute -g on R_3
$R_1 \longleftarrow R_1 + R_2 \cdot R_3$

R_1	R_2	R_3
a	b	c
a	b+f	c
a-(b+f)c	b+f	c
a-(b+f)c	b+f	c+g
a-(b+f)c+(b+f)(c+g)	b+f	c+g

here $\mathcal{R}$ is not necessarily commutative

Algebraic formula over ring $\mathcal{R}$

In fact, any algebraic formula f can be computed **cleanly** with 3 registers

- trivially if f is an input or a constant
- Suppose it holds for f and g, what about f·g

Compute f on R_2
$R_1 \longleftarrow R_1 - R_2 \cdot R_3$
Compute g on R_3
$R_1 \longleftarrow R_1 + R_2 \cdot R_3$
Compute -f on R_2
$R_1 \longleftarrow R_1 - R_2 \cdot R_3$
Compute -g on R_3
$R_1 \longleftarrow R_1 + R_2 \cdot R_3$

R_1	R_2	R_3
a	b	c
a	b+f	c
a-(b+f)c	b+f	c
a-(b+f)c	b+f	c+g
a+(b+f)g	b+f	c+g

Algebraic formula over ring $\mathcal{R}$

In fact, any algebraic formula f can be computed **cleanly** with 3 registers

- trivially if f is an input or a constant
- Suppose it holds for f and g, what about f·g

Compute f on R_2
$R_1 \longleftarrow R_1 - R_2 \cdot R_3$
Compute g on R_3
$R_1 \longleftarrow R_1 + R_2 \cdot R_3$
Compute -f on R_2
$R_1 \longleftarrow R_1 - R_2 \cdot R_3$
Compute -g on R_3
$R_1 \longleftarrow R_1 + R_2 \cdot R_3$

R_1	R_2	R_3
a+(b+f)g	b+f	c+g
a+(b+f)g	b	c+g
a+(b+f)g-b(c+g)	b	c+g

Algebraic formula over ring $\mathcal{R}$

In fact, any algebraic formula f can be computed **cleanly** with 3 registers

- trivially if f is an input or a constant
- Suppose it holds for f and g, what about f·g

Compute f on R_2
$R_1 \longleftarrow R_1 - R_2 \cdot R_3$
Compute g on R_3
$R_1 \longleftarrow R_1 + R_2 \cdot R_3$
Compute -f on R_2
$R_1 \longleftarrow R_1 - R_2 \cdot R_3$
Compute -g on R_3
$R_1 \longleftarrow R_1 + R_2 \cdot R_3$

R_1	R_2	R_3
a+(b+f)g	b+f	c+g
a+(b+f)g	b	c+g
a+fg-bc	b	c+g
a+fg-bc	b	c
a+fg	b	c

Algebraic formula over ring $\mathcal{R}$

In fact, any algebraic formula f can be computed **cleanly** with 3 registers

- trivially if f is an input or a constant
- Suppose it holds for f and g, what about f·g

Compute f on R_2
$R_1 \longleftarrow R_1 - R_2 \cdot R_3$
Compute g on R_3
$R_1 \longleftarrow R_1 + R_2 \cdot R_3$
Compute -f on R_2
$R_1 \longleftarrow R_1 - R_2 \cdot R_3$
Compute -g on R_3
$R_1 \longleftarrow R_1 + R_2 \cdot R_3$

R_1	R_2	R_3
a+fg	b	c

For depth d formula, we get a reversible program of length $O(4^d)$

Algebraic formula over ring $\mathcal{R}$

Depth d algebraic formula can be computed by reversible program of length $O(4^d)$ with 3 registers

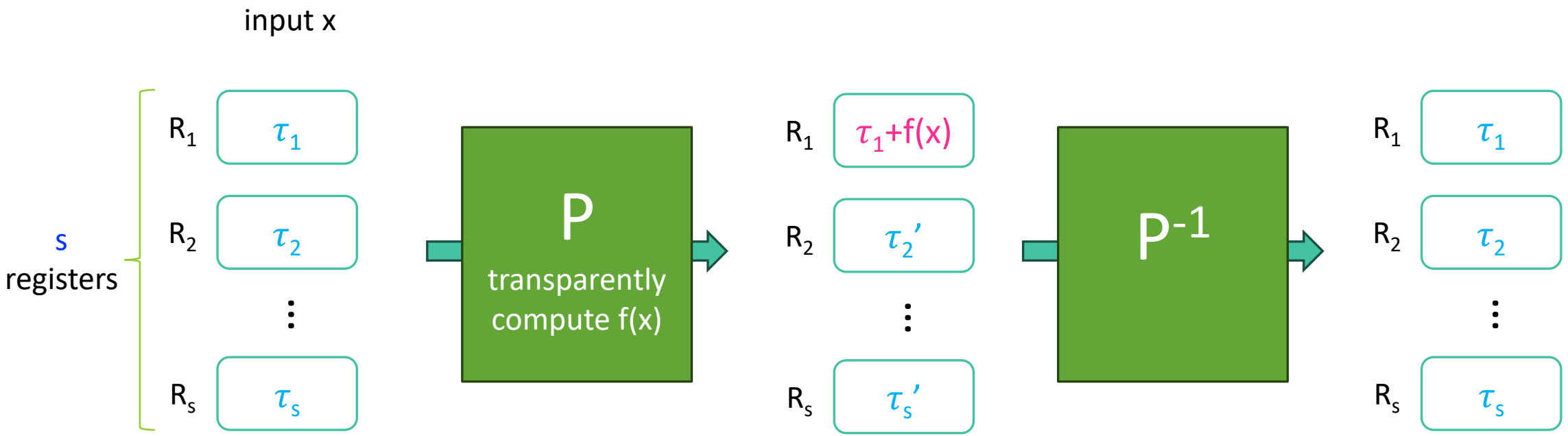
Any formula of length n is equivalent to a formula of depth O(log n) *[Brent 1974]*

Thus, formula of length n can be computed by reversible program of length poly(n) with 3 registers

Reversible program ⟹ Catalytic space

CSPACE[s(n),c(n)]: O(s(n)) space of work tape and O(c(n)) space of the auxiliary tape

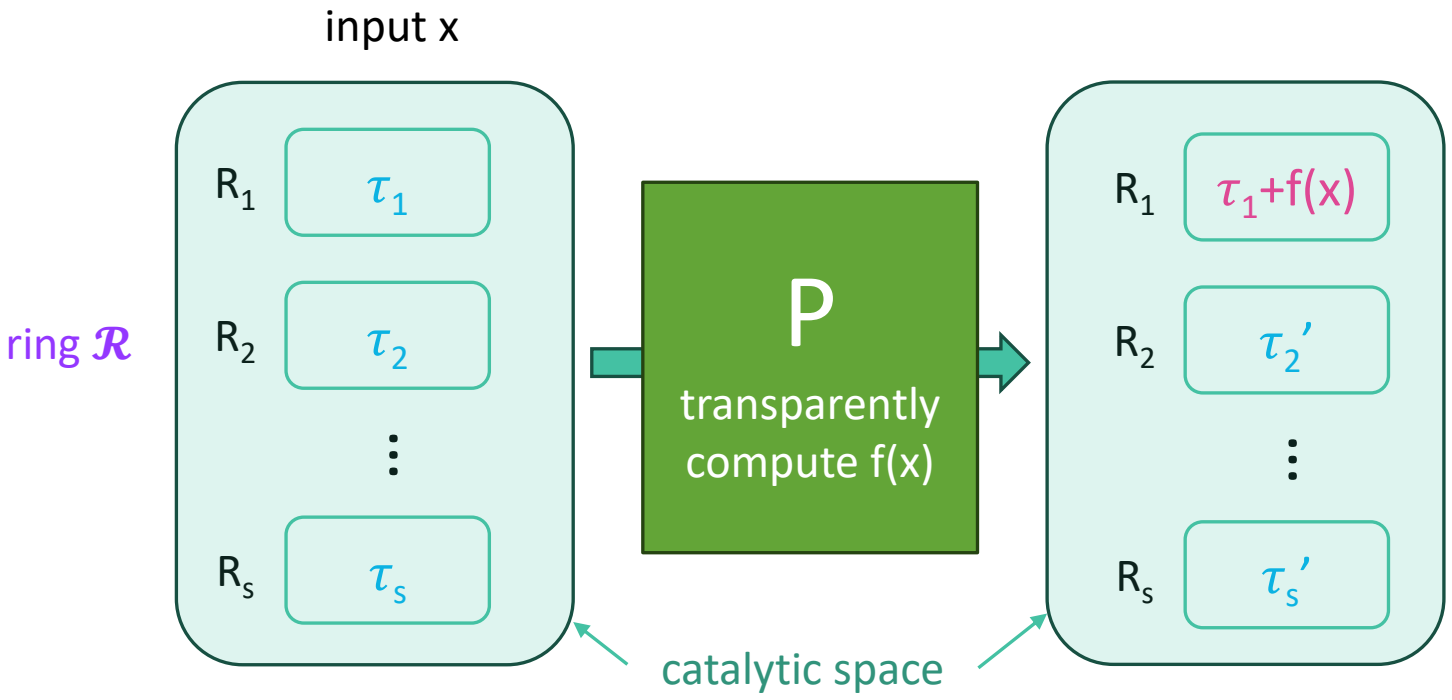
CL=CSPACE[log n, poly(n)]: logspace of work tape and polynomial space of the auxiliary tape



Reversible program $\Longrightarrow$ Catalytic space

CSPACE[s(n),c(n)]: O(s(n)) space of work tape and O(c(n)) space of the auxiliary tape

CL=CSPACE[log n, poly(n)]: logspace of work tape and polynomial space of the auxiliary tape



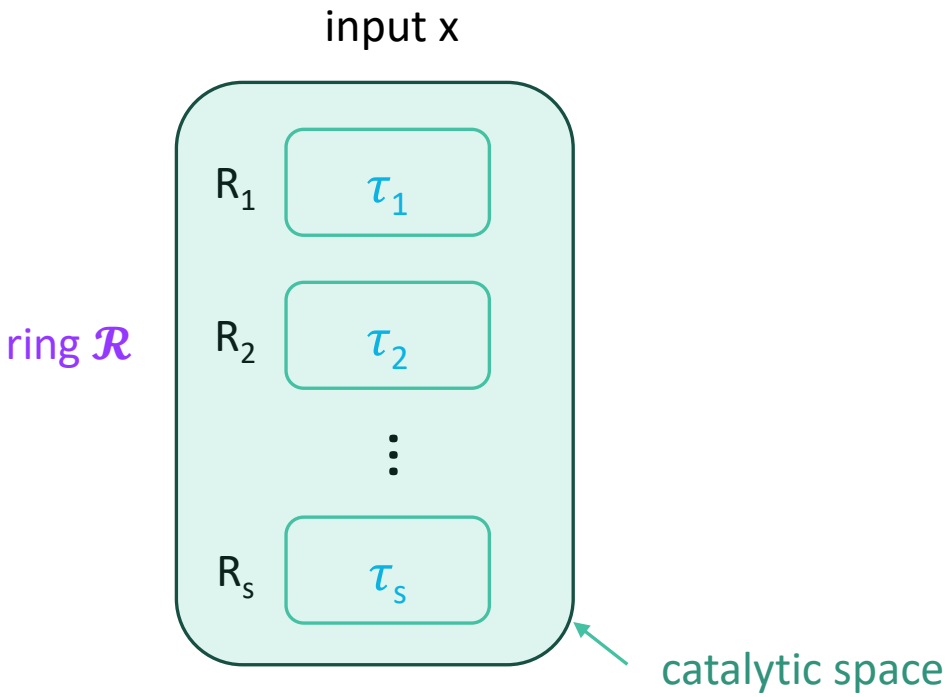
For CL, we need:

- operations in $\mathcal{R}$ are easy to compute
  in fact , it is a sequence $\{\mathcal{R}_n\}$
- #register is polynomial in |x|
- **P**: poly length, logspace uniform:
  given input x and index i, performs i^{th} *step of P in logspace*
- we need to store content τ_1, so each register takes logspace

Reversible program $\Longrightarrow$ Catalytic space

CSPACE[s(n),c(n)]: O(s(n)) space of work tape and O(c(n)) space of the auxiliary tape

CL=CSPACE[log n, poly(n)]: logspace of work tape and polynomial space of the auxiliary tape



It will be easy that $|\mathcal{R}|=2^i$ for some integer i

then each register takes i bits

What about $|\mathcal{R}|$ is not a power of 2

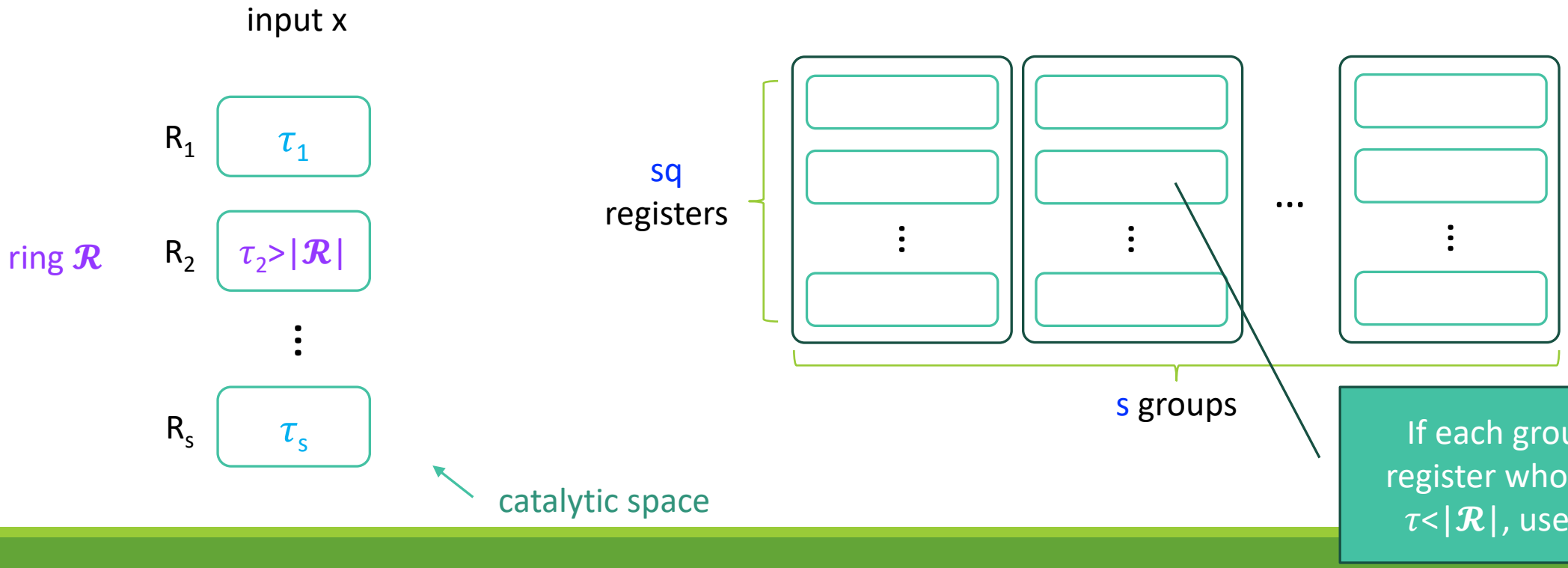
initial contents may be not in $\mathcal{R}$

Reversible program ⟹ Catalytic space

What about $|\mathcal{R}|$ is not a power of 2

initial contents may be not in $\mathcal{R}$

let $q = \lceil \log_2 |\mathcal{R}| \rceil$, we can use $s^2 q$ registers of q bits

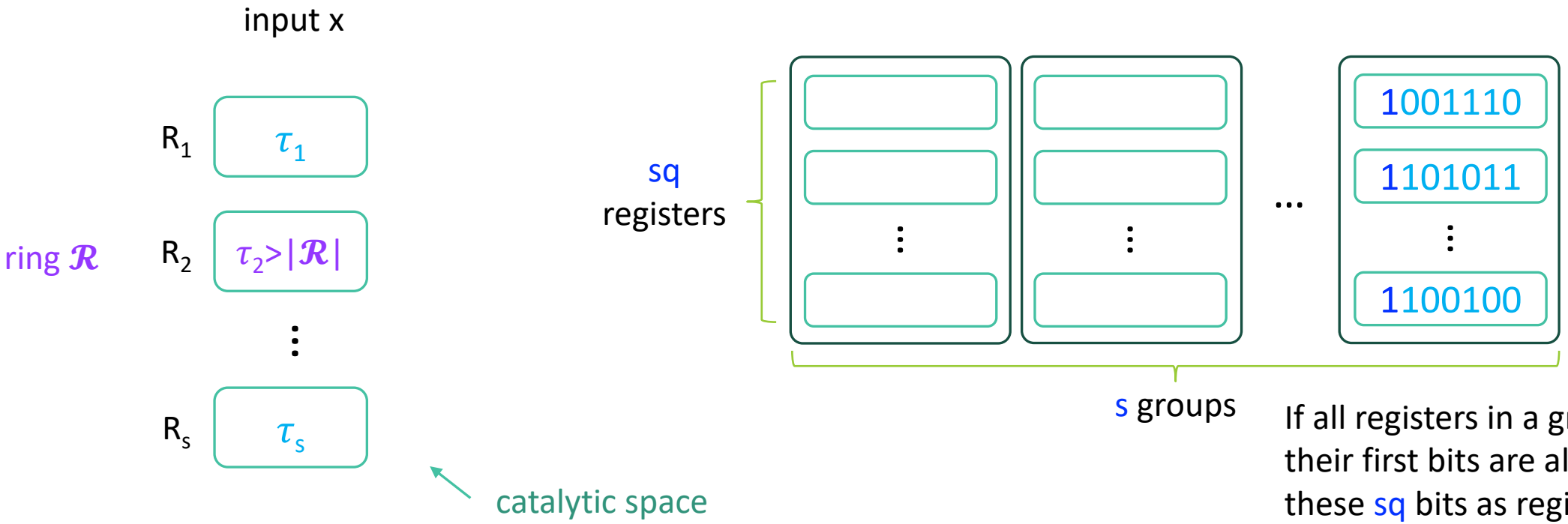


Reversible program ⟹ Catalytic space

What about $|\mathcal{R}|$ is not a power of 2

initial contents may be not in $\mathcal{R}$

let $q = \lceil \log_2 |\mathcal{R}| \rceil$, we can use s^2q registers of q bits



If all registers in a group is $\geq |\mathcal{R}|$, their first bits are all 1, so just use these sq bits as registers

CL: catalytic logspace

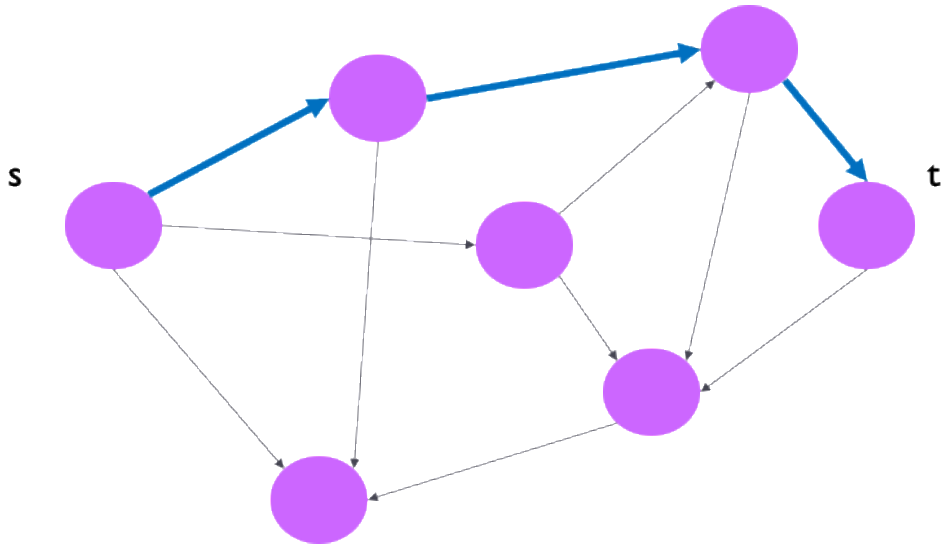
Is CL larger than L?

We will show that NL ⊆ CL

- we don't know whether L=NL

Proof

- PATH ∈ CL
- PATH is *NL-complete*



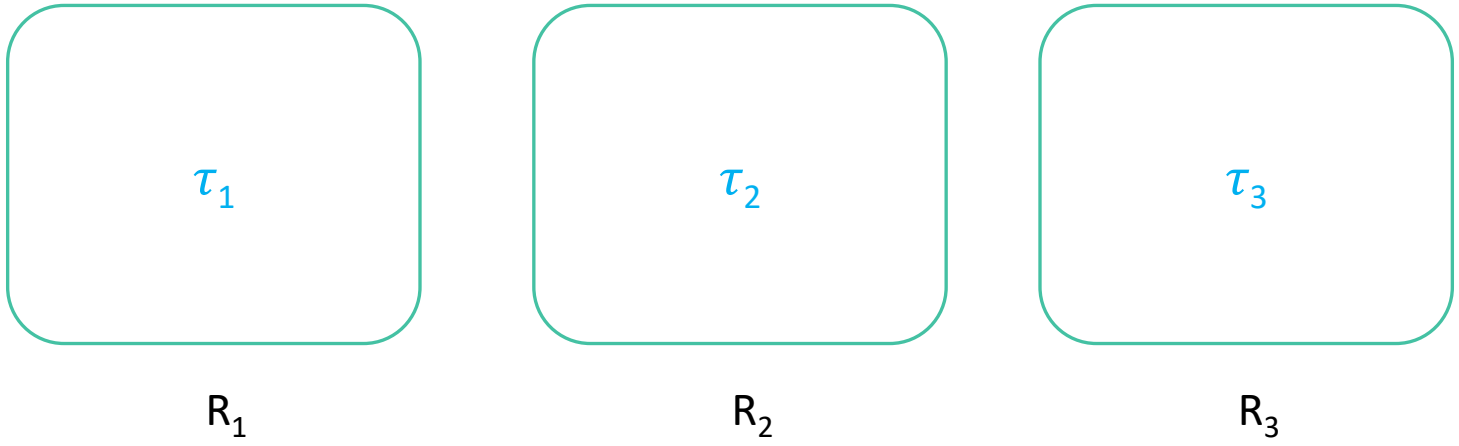
reachability in directed graph

PATH ∈ CL

A = adjacency matrix (with self-loops)

compute A^n

- using $O(n^2)$ registers, we can cleanly compute matrix multiplication

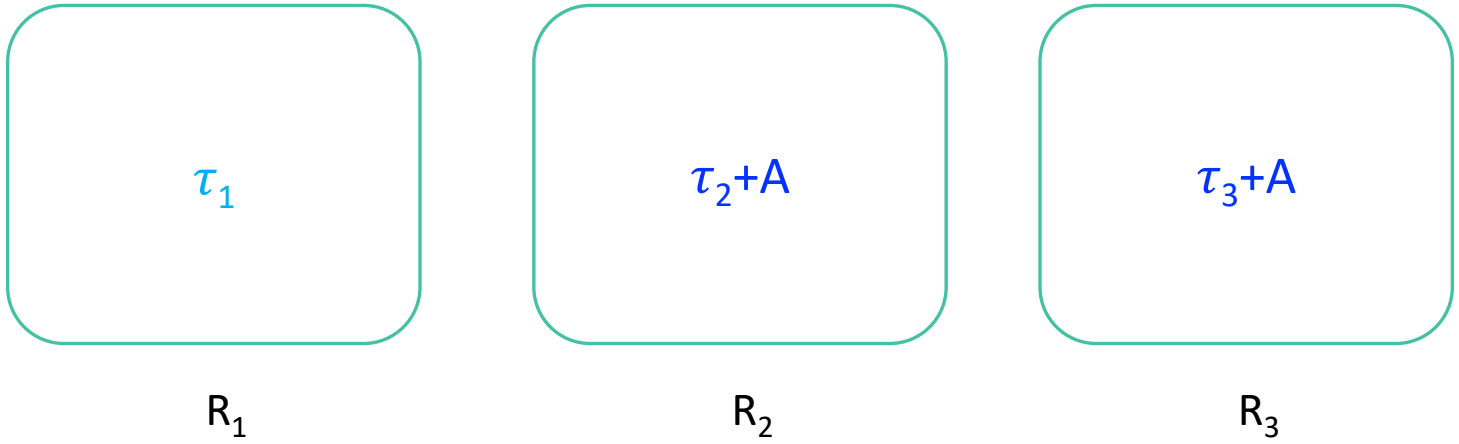


PATH ∈ CL

A = adjacency matrix (with self-loops)

compute A^n

- using $O(n^2)$ registers, we can cleanly compute matrix multiplication



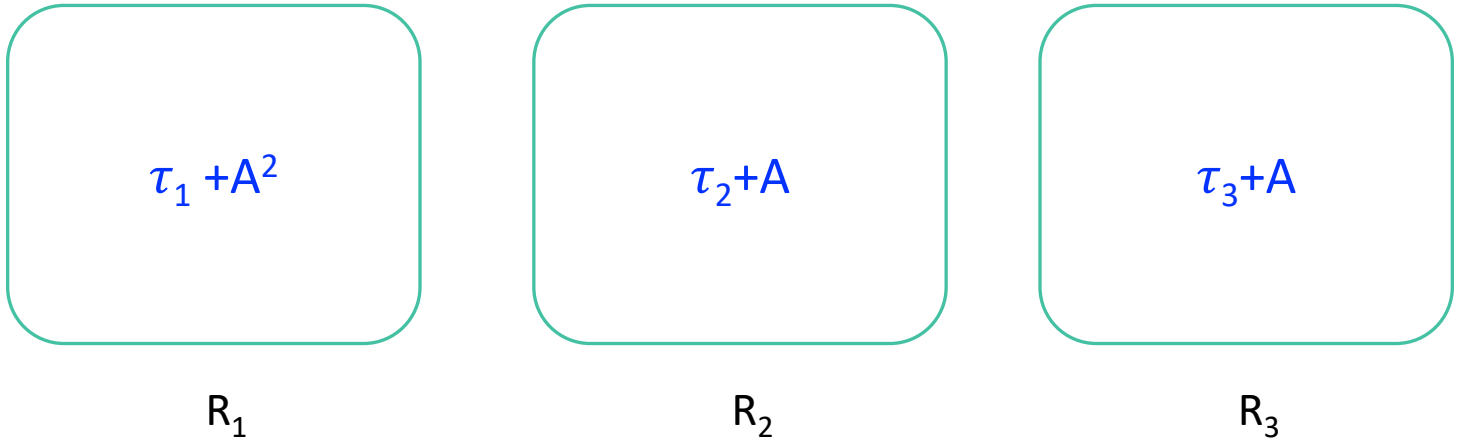
PATH ∈ CL

A = adjacency matrix (with self-loops)

compute A^n

- using $O(n^2)$ registers, we can cleanly compute matrix multiplication
- polynomial-size catalytic tape, program is logspace uniform

$(A^n)_{s,t}$ is the result



PATH $\in$ CL

A = adjacency matrix (with self-loops)

compute A^n

- using $O(n^2)$ registers, we can cleanly compute matrix multiplication
- polynomial-size catalytic tape, program is logspace uniform
- we can compute A^n in $\mathcal{R}$, but what is $\mathcal{R}$?

Elements in A^n can be as large n^n,

- Just need to check whether $(A^n)_{s,t} = 0$
- we can run this algorithm on $\mathbb{Z}_k = \{0,1,\ldots,k-1\}$ for all $k=2,\ldots,O(n \cdot \log n)$

CL: catalytic logspace

We know:

$$NC^1 \subseteq L \subseteq NL \subseteq AC^1 \subseteq ACC^1 \subseteq TC^1 \subseteq CL$$

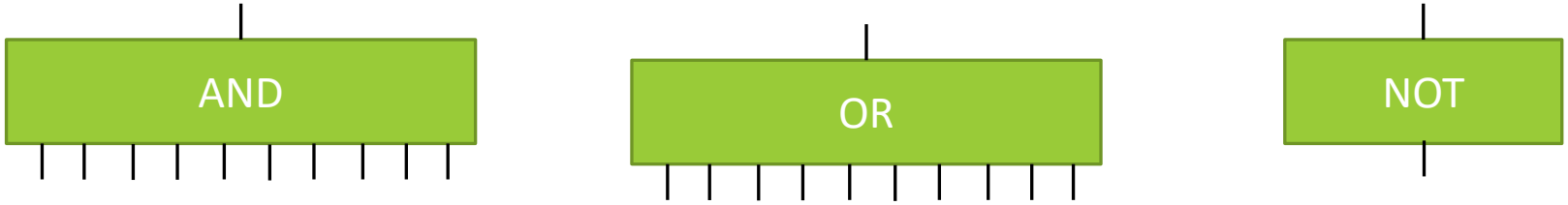
[Buhrman, Cleve, Koucký, Loff, Speelman 14]

CL: catalytic logspace

We know:

$$NC^1 \subseteq L \subseteq NL \subseteq AC^1 \subseteq ACC^1 \subseteq TC^1 \subseteq CL$$

AC^1: O(log n)-depth polynomial-size circuits $\{C_n\}$ with unbounded fan-in gates

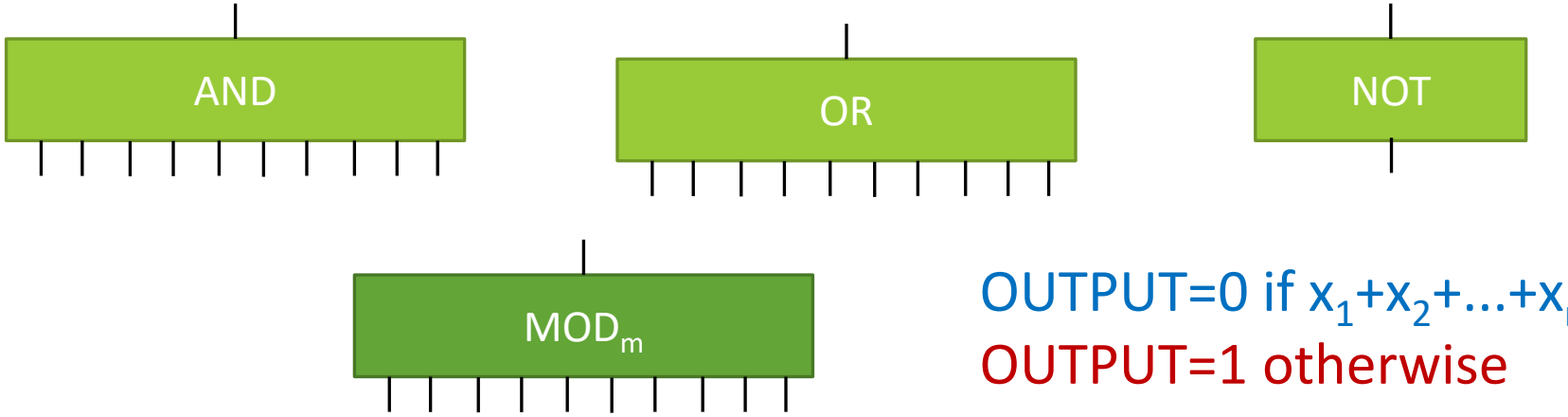


CL: catalytic logspace

We know:

$$NC^1 \subseteq L \subseteq NL \subseteq AC^1 \subseteq ACC^1 \subseteq TC^1 \subseteq CL$$

ACC^1: O(log n)-depth polynomial-size circuits $\{C_n\}$ with unbounded fan-in AND, OR and MOD_m gates



OUTPUT=0 if $x_1+x_2+\ldots+x_n \equiv 0 \pmod p$

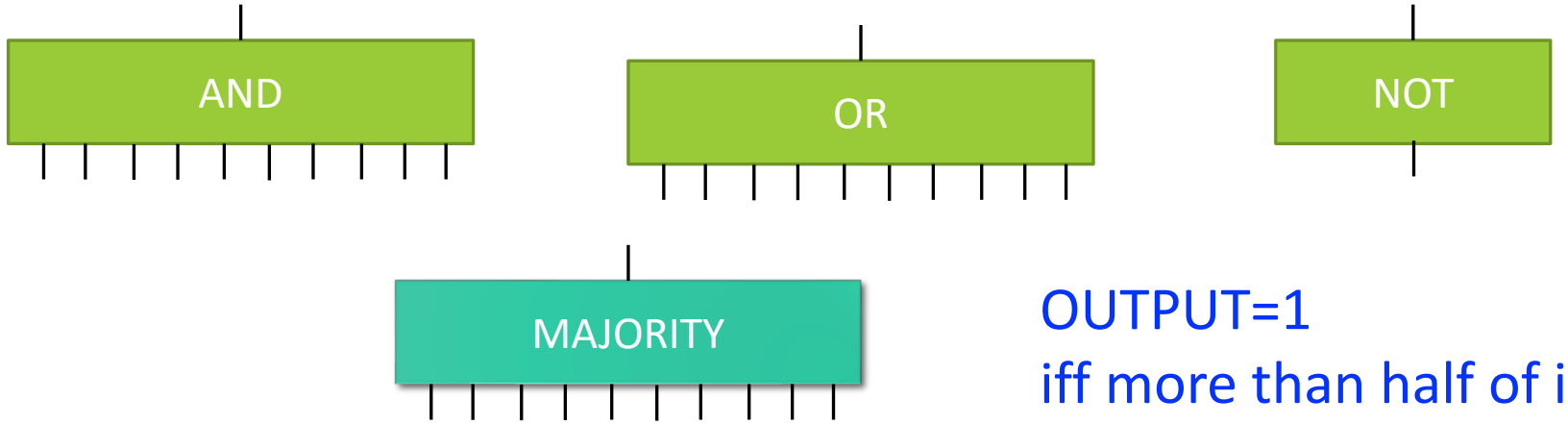
OUTPUT=1 otherwise

CL: catalytic logspace

We know:

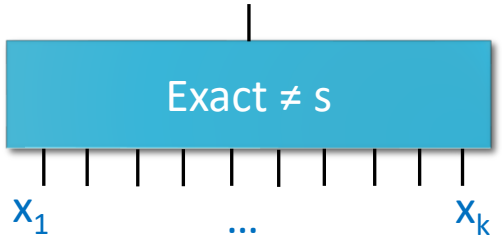
$$NC^1 \subseteq L \subseteq NL \subseteq AC^1 \subseteq ACC^1 \subseteq TC^1 \subseteq CL$$

TC^1: O(log n)-depth polynomial-size circuits $\{C_n\}$ with unbounded fan-in AND, OR and MAJORITY gates

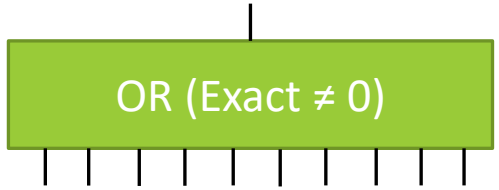


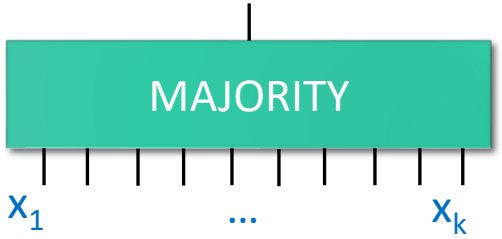
OUTPUT=1
iff more than half of inputs are 1

TC^1: only using exact-value gate

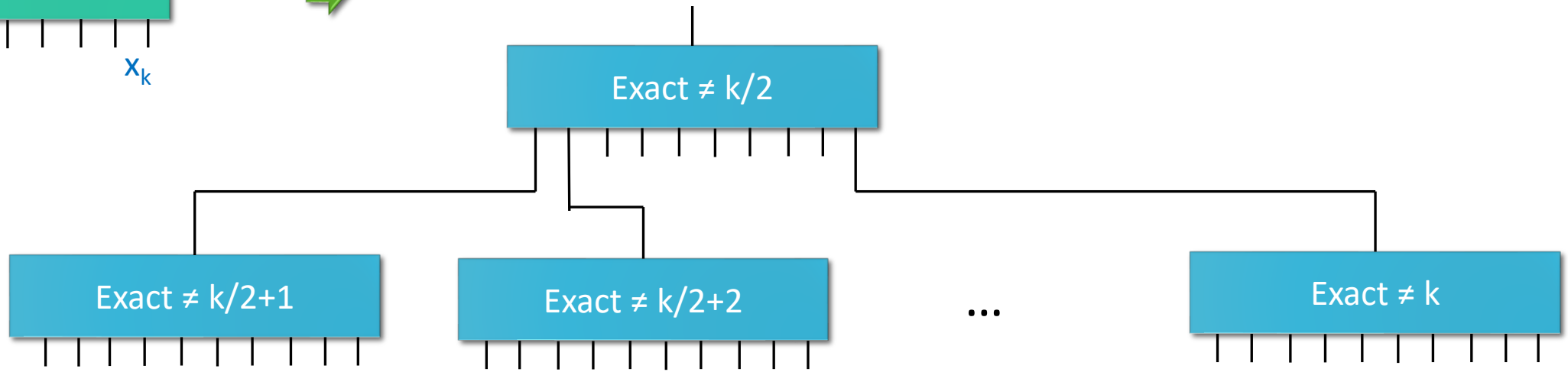


OUTPUT=1 iff $x_1+x_2+\ldots+x_k \neq s$





(k even)



Similar for NOT, AND, MOD gates

Powering lemma

If function f(x) can be transparently computed into register R_f by **P**, then $[f(x)]^k$ can be transparently computed into R_0

- with O(k) more registers and polynomial more instructions

For i ⟵ 1 to k do

$$R_0 \leftarrow R_0 + (-1)^{i-1}\binom{k}{i} \cdot R_i (R_f)^{k-i}$$

P^{-1} : Compute -f on R_f

For i ⟵ 1 to k do

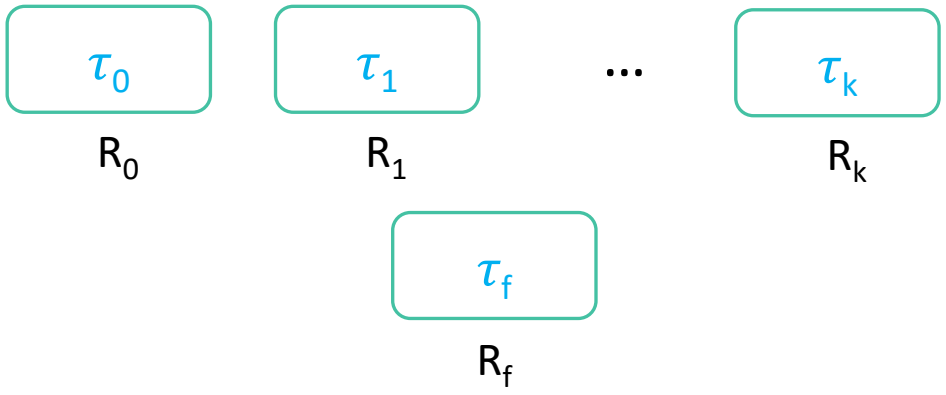
$$R_i \leftarrow R_i + (R_f)^i$$

P: Compute f on R_f

$$R_0 \leftarrow R_0 + (R_f)^k$$

For i ⟵ 1 to k do

$$R_0 \leftarrow R_0 - (-1)^{i-1}\binom{k}{i} \cdot R_i (R_f)^{k-i}$$



Powering lemma

For i ⟵ 1 to k do

$R_0 \longleftarrow R_0 + (-1)^{i-1}\binom{k}{i} \cdot R_i (R_f)^{k-i}$

P^{-1} : Compute -f on R_f

For i ⟵ 1 to k do

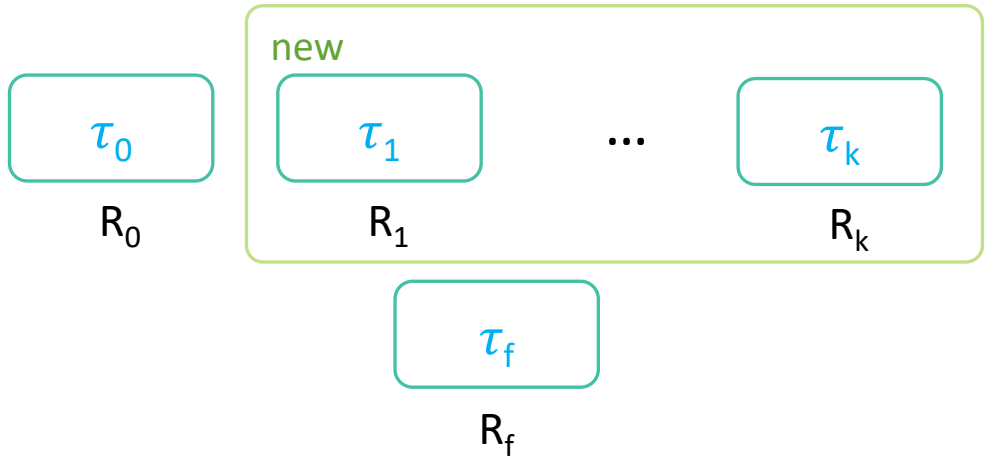
$R_i \longleftarrow R_i + (R_f)^i$

P: Compute f on R_f

$R_0 \longleftarrow R_0 + (R_f)^k$

For i ⟵ 1 to k do

$R_0 \longleftarrow R_0 - (-1)^{i-1}\binom{k}{i} \cdot R_i (R_f)^{k-i}$



$$R_0 = \tau_0 + \sum_{i=1}^{k}\left[(-1)^{i-1}\binom{k}{i} \cdot \tau_i (\tau_f)^{k-i}\right]$$

$$R_f = \tau_f - f(x)$$

$$R_i = \tau_i + (\tau_f - f(x))^i$$

$$R_f = \tau_f$$

$$R_0 = \tau_0 + \sum_{i=1}^{k}\left[(-1)^{i-1}\binom{k}{i} \cdot \tau_i (\tau_f)^{k-i}\right] + (\tau_f)^k$$

$$R_0$$
$$= \tau_0 + \sum_{i=1}^{k}\left[(-1)^{i-1}\binom{k}{i} \cdot \tau_i (\tau_f)^{k-i}\right] + (\tau_f)^k$$
$$- \sum_{i=1}^{k}\left[(-1)^{i-1}\binom{k}{i} \cdot \left(\tau_i + (\tau_f - f(x))^i\right)(\tau_f)^{k-i}\right]$$

Powering lemma

For i ⟵ 1 to k do

$R_0 \longleftarrow R_0 + (-1)^{i-1}\binom{k}{i} \cdot R_i (R_f)^{k-i}$

P^{-1} : Compute -f on R_f

For i ⟵ 1 to k do

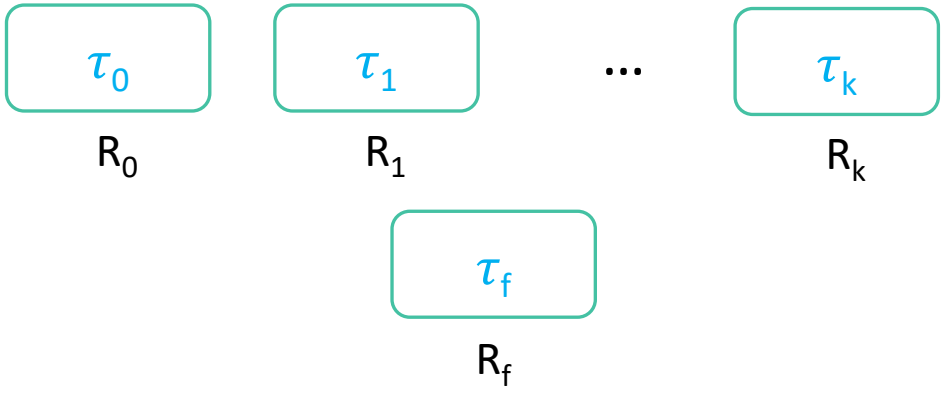
$R_i \longleftarrow R_i + (R_f)^i$

P: Compute f on R_f

$R_0 \longleftarrow R_0 + (R_f)^k$

For i ⟵ 1 to k do

$R_0 \longleftarrow R_0 - (-1)^{i-1}\binom{k}{i} \cdot R_i (R_f)^{k-i}$

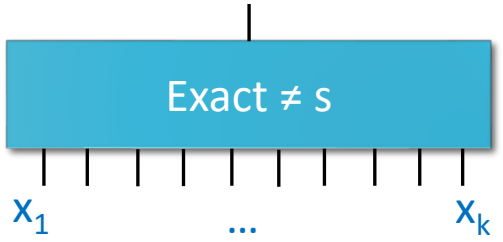


$$
\begin{aligned}
& R_0 \\
&= \tau_0 + \sum_{i=1}^{k} \left[(-1)^{i-1} \binom{k}{i} \cdot \tau_i (\tau_f)^{k-i} \right] + (\tau_f)^k \\
&- \sum_{i=1}^{k} \left[(-1)^{i-1} \binom{k}{i} \cdot \left(\tau_i + \left(\tau_f - f(x) \right)^i \right) (\tau_f)^{k-i} \right] \\
&= \tau_0 + \sum_{i=1}^{k} \left[(-1)^{i} \binom{k}{i} \cdot \left(\tau_f - f(x) \right)^i (\tau_f)^{k-i} \right] + (\tau_f)^k \\
&= \tau_0 + \sum_{i=0}^{k} \left[(-1)^{i} \binom{k}{i} \cdot \left(\tau_f - f(x) \right)^i (\tau_f)^{k-i} \right] \\
&= \tau_0 + \sum_{i=0}^{k} \left[\binom{k}{i} \cdot \left(f(x) - \tau_f \right)^i (\tau_f)^{k-i} \right] = \tau_0 + (f(x))^k
\end{aligned}
$$

Exact-value

In field $\mathcal{R}=\mathbb{Z}_p$ where p is a prime, if $f_i(x)$ can be transparently computed in registers R_i for i=1,…,k, then $[\![\sum_{i=1}^{k} f_i(x) \neq s]\!]$ can be computed transparently

- with O(p) more registers
- $[\![\sum_{i=1}^{k} f_i(x) \neq s]\!] = 1$ if $\sum_{i=1}^{k} f_i(x) \neq s$, otherwise it is 0



OUTPUT=1 iff $x_1+x_2+\ldots+x_k \neq s$

Exact-value

In field $\mathcal{R}=\mathbb{Z}_p$ where p is a prime, if $f_i(x)$ can be transparently computed in registers R_i for i=1,…,k, then $[\![\sum_{i=1}^{k} f_i(x) \neq s]\!]$ can be computed transparently

- with O(p) more registers
- $[\![\sum_{i=1}^{k} f_i(x) \neq s]\!] = 1$ if $\sum_{i=1}^{k} f_i(x) \neq s$, otherwise it is 0

Transparently compute $\sum_{i=1}^{k} f_i(x) - s$ on R_0

$R_0 \longleftarrow R_0 - R_1 - \ldots - R_k$
For i $\longleftarrow$ 1 to k do
    Compute $f_i(x)$ on R_i
$R_0 \longleftarrow R_0 + R_1 + \ldots + R_k - s$

Use powering lemma to compute $\left(\sum_{i=1}^{k} f_i(x) - s\right)^{p-1}$

by Fermat's little theorem

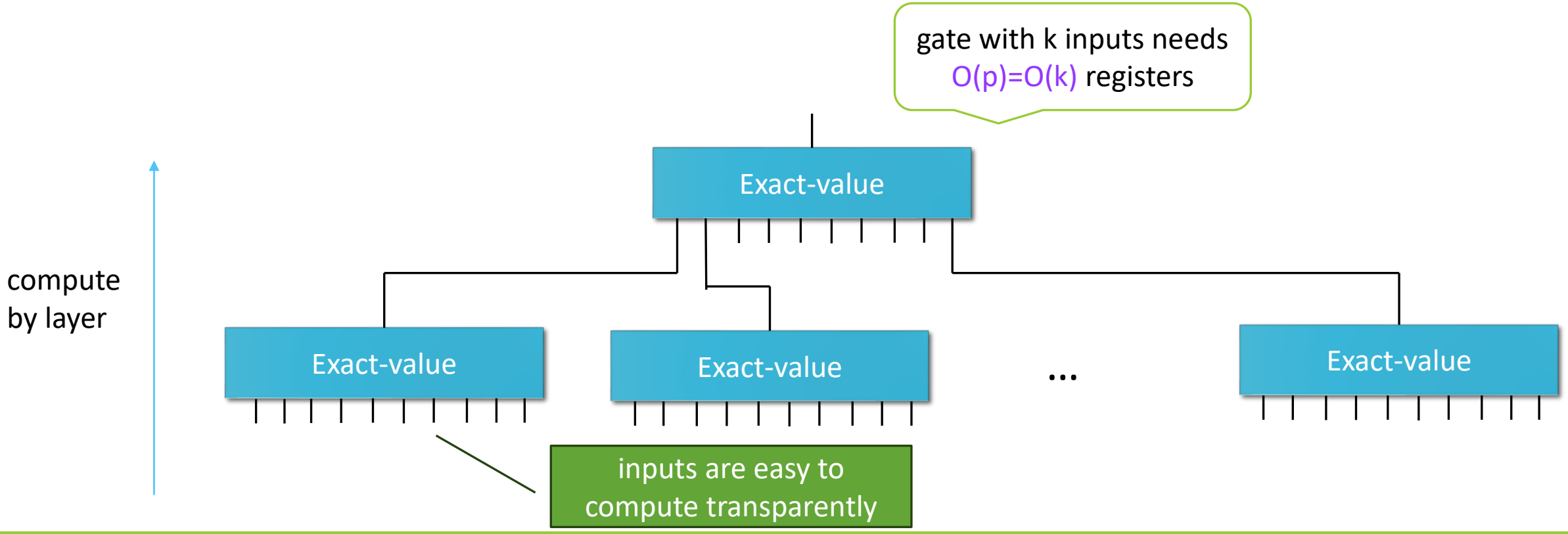
$$\left(\sum_{i=1}^{k} f_i(x) - s\right)^{p-1} = \begin{cases} 0, & \text{if } \sum_{i=1}^{k} f_i(x) = s \\ 1, & \text{otherwise} \end{cases}$$

here we need p>k

In field $\mathcal{R}=\mathbb{Z}_p$ where p is a prime, if $f_i(x)$ can be transparently computed in registers R_i for i=1,…,k, then $[\![\sum_{i=1}^{k} f_i(x) \neq s]\!]$ can be computed transparently with O(p) more registers

TC^1: *d=O(log n)*-depth polynomial-size circuits $\{C_n\}$ with unbounded fan-in Exact-value gates

- make it layered: each gate at level $\boldsymbol{l}$ takes as its inputs gates at level $\boldsymbol{l}-1$
- polynomial of registers, each register takes O(log p)=O(log n) bits



$TC^1 \in CL$

TC^1: *d=O(log n)*-depth polynomial-size circuits $\{C_n\}$ with unbounded fan-in Exact-value gates

- polynomial of registers, each register takes O(log p)=O(log n) bits
- logspace reduction: not hard to see
- so **$TC^1 \in CL$**

For CL, we need:

- operations in $\mathcal{R}$ are easy to compute
  in fact , it is a sequence $\{\mathcal{R}_n\}$
- #register is polynomial in |x|
- **P**: poly length, logspace uniform:
  given input x and index i, performs i^{th} *step of P in logspace*
- we need to store content τ_1, so each register takes logspace

CL: catalytic logspace

CL=CSPACE[log n, poly(n)]: logspace of work tape and polynomial space of the auxiliary tape

We know:

$$NC^1 \subseteq L \subseteq NL \subseteq AC^1 \subseteq ACC^1 \subseteq TC^1 \subseteq CL$$

Is $CL \subseteq P$?

- open

but we can show $CL \subseteq ZPP$ *[Buhrman, Cleve, Koucký, Loff, Speelman 14]*

CL: catalytic logspace

CL=CSPACE[s(n), c(n)], s(n)=O(log n), c(n)=poly(n)

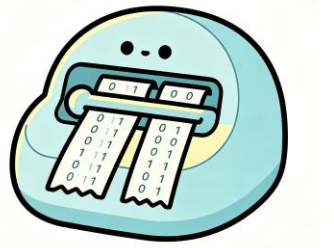
work tape

auxiliary tape

x

#states: $O(2^{s(n)})$, polynomial

#states: $O(2^{c(n)})$, exponential

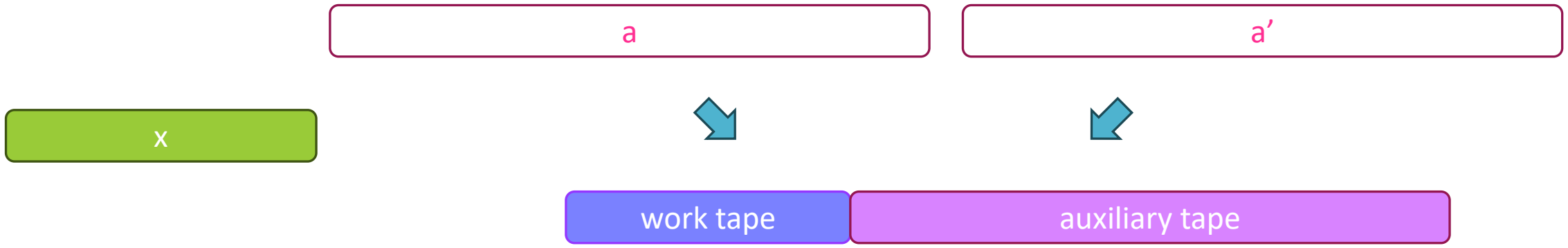


CL: catalytic logspace

CL=CSPACE[s(n), c(n)], s(n)=O(log n), c(n)=poly(n)

given x, two different initial auxiliary tape contents a,a’

- they cannot lead to the same configuration, otherwise they cannot come back to different a,a’



CL=CSPACE[s(n), c(n)], s(n)=O(log n), c(n)=poly(n)

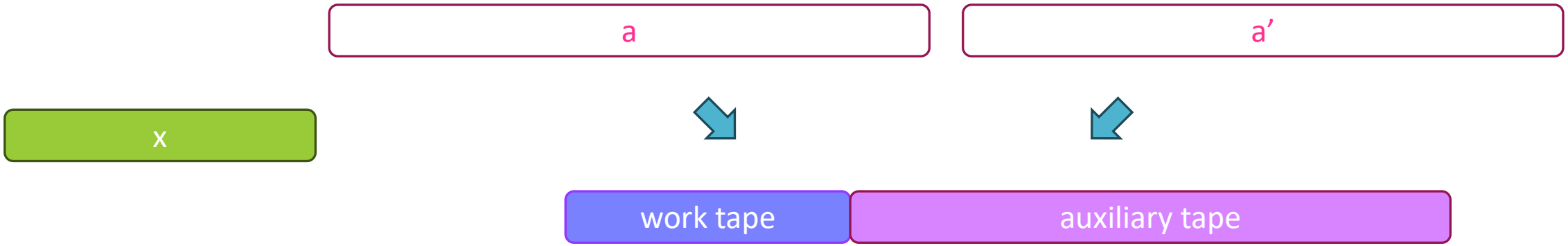
given x, two different initial auxiliary tape contents a,a’

- they cannot lead to the same configuration, otherwise they cannot come back to different a,a’
- total number of configurations: $O(2^{s(n)+c(n)})$
- T(x,a): time for input x and initial auxiliary tape contents a

$$\sum_a T(x,a) \leq O\left(2^{s(n)+c(n)}\right)$$

#initial a: $O(2^{c(n)})$

$$\mathbb{E}_a T(x,a) \leq O\left(2^{s(n)}\right)$$



CL=CSPACE[s(n), c(n)], s(n)=O(log n), c(n)=poly(n)

given x, two different initial auxiliary tape contents a,a'

- they cannot lead to the same configuration, otherwise they cannot come back to different a,a'
- total number of configurations: $O(2^{s(n)+c(n)})$
- T(x,a): time for input x and initial auxiliary tape contents a

$$\sum_a T(x,a) \leq O\left(2^{s(n)+c(n)}\right)$$

#initial a: $O(2^{c(n)})$

$$\mathbb{E}_a T(x,a) \leq O\left(2^{s(n)}\right)$$

so random a can give expected polynomial time for s(n)=O(log n)

$$CL \subseteq ZPP$$

CL and P

CL vs P is still open

CLP: the subclass of CL where the machine is additionally required to run in polynomial time

Recent results:

CLP = CL ∩ P *[Cook, Li, Mertz, Pyne 25]*

BPL ⊆ CL *[Pyne 24]* (bounded-error probabilistic logspace)

Bipartite Matching ⊆ CL *[Agarwala, Mertz 25]*

Tree Evaluation

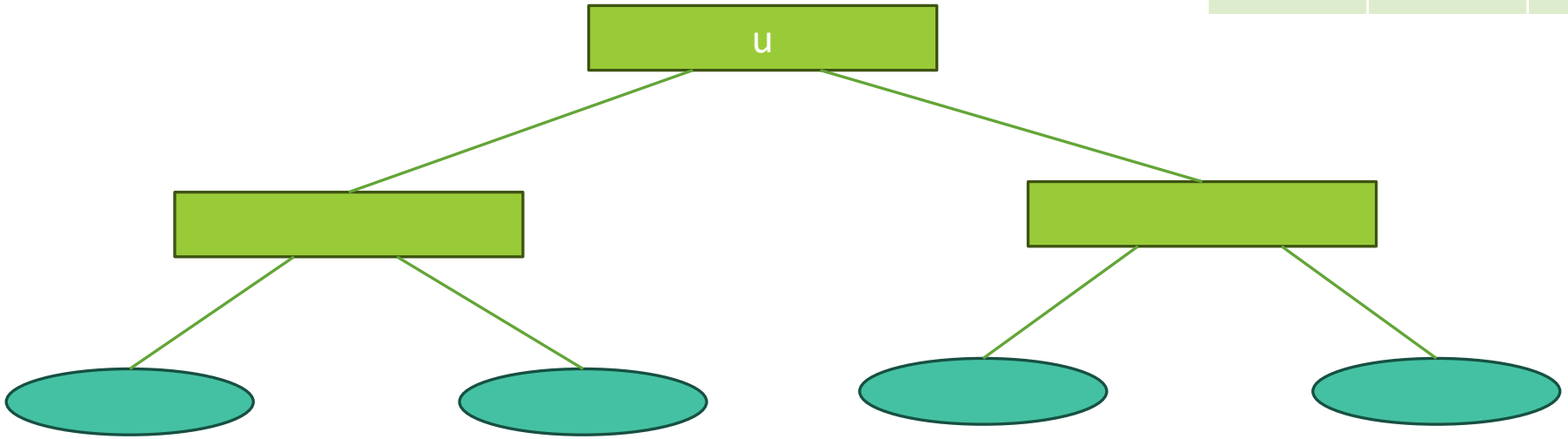
Complete binary tree

- height h, alphabet [k]={1,2,…,k}

given by a table of k^2 values

L	R	output
1	1	
	…	
k	k	

each node u has a function
$f_u: [k] \times [k] \rightarrow [k]$



each leaf has an input value in [k]

Tree Evaluation

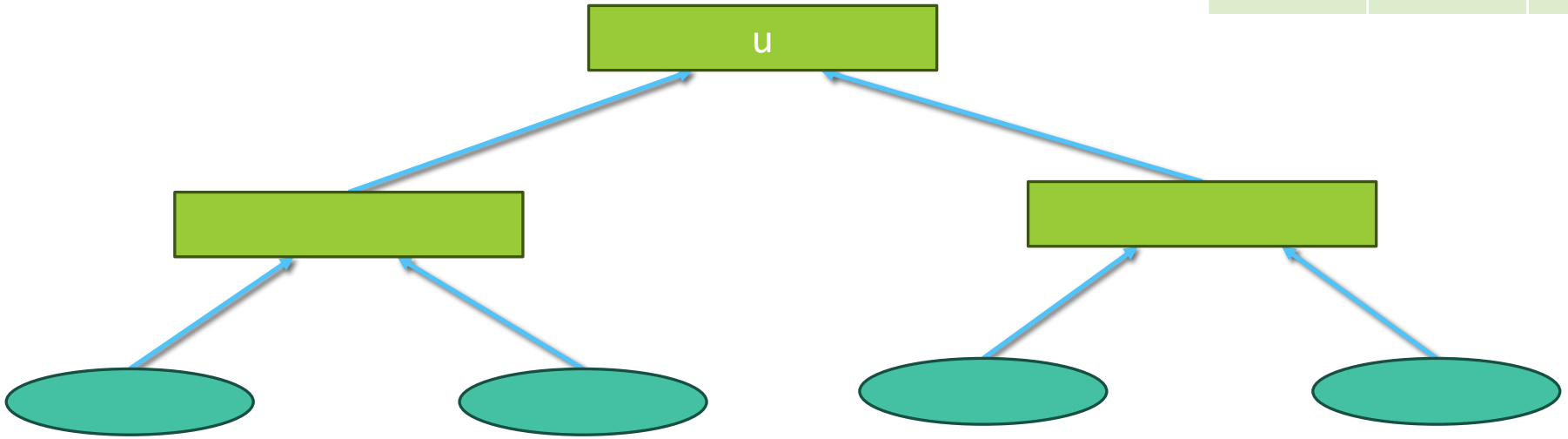
Complete binary tree

- height h, alphabet [k]={1,2,…,k}
- compute the values of every node from its children until the root
- input size: $2^h \cdot poly(k)$

given by a table of k^2 values

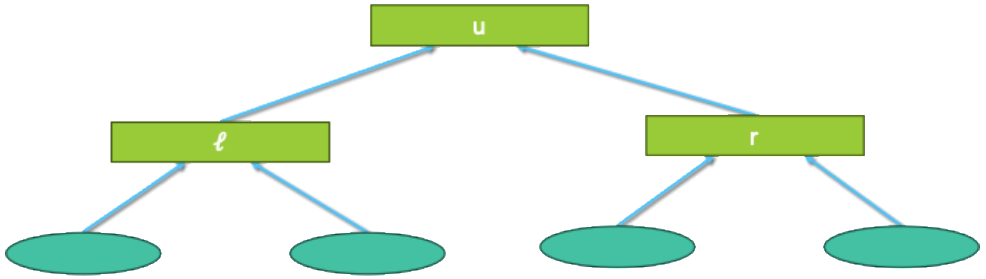
L	R	output
1	1	
…		
k	k	

each node u has a function
$f_u: [k] \times [k] \to [k]$



each leaf has an input value in [k]

Tree Evaluation



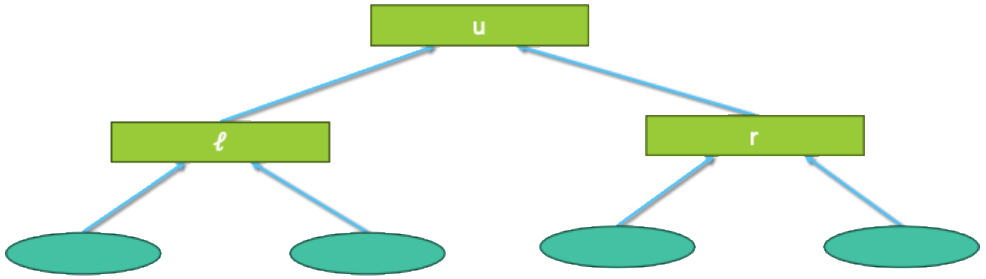
Complete binary tree

- height h, alphabet [k]={1,2,…,k}
- each leaf has an input value in [k], each node u has a function f_u: [k]×[k]⟶[k]
- compute the values of every node from its children until the root
- input size: $2^h \cdot poly(k)$

Easy to compute, but what about space?

```
Value(u) {
    if (u is a leaf), return input(u)
    a ⟵ Value(ℓ)
    b ⟵ Value(r)
    return f_u(a,b)
}
```

Tree Evaluation



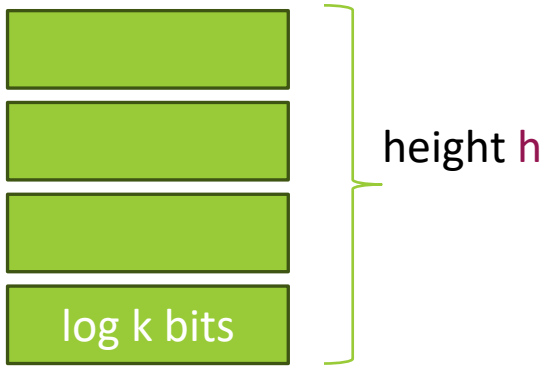
Complete binary tree

- height h, alphabet [k]={1,2,…,k}
- each leaf has an input value in [k], each node u has a function f_u: [k]×[k]⟶[k]
- compute the values of every node from its children until the root
- input size: $2^h \cdot poly(k)$

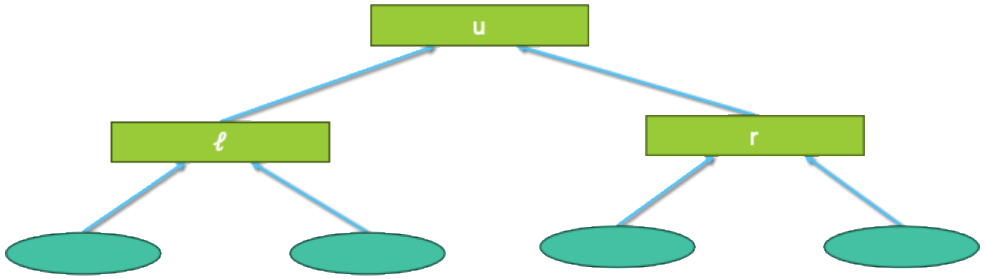
Easy to compute, but what about space?

```
Value(u) {
    if (u is a leaf), return input(u)
    a ⟵ Value(ℓ)
    b ⟵ Value(r)
    return f_u(a,b)
}
```

when calling Value(r), we need to push a to the stack



Tree Evaluation



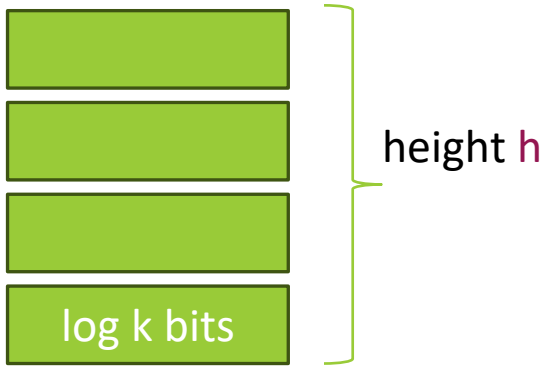
Complete binary tree

- height h, alphabet [k]={1,2,…,k}
- each leaf has an input value in [k], each node u has a function f_u: [k]×[k]⟶[k]
- compute the values of every node from its children until the root
- input size: $2^h \cdot poly(k)$

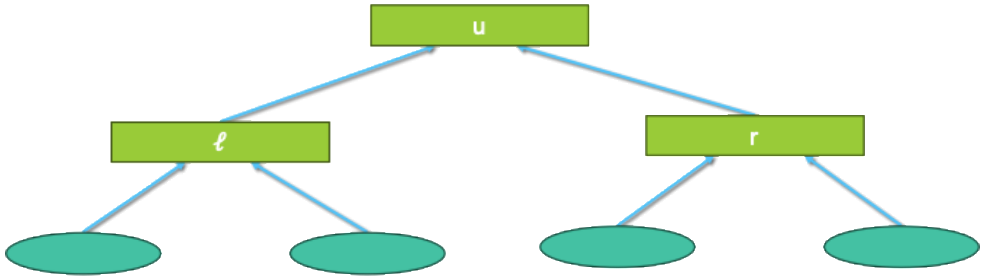
Easy to compute, but what about space?

- trivial space: O(h·log k)
- log(*input size*)=O(h+log k)
- so TreeEval ∈ L^2

- (considered to be a separating example for L and P)



Tree Evaluation



Complete binary tree

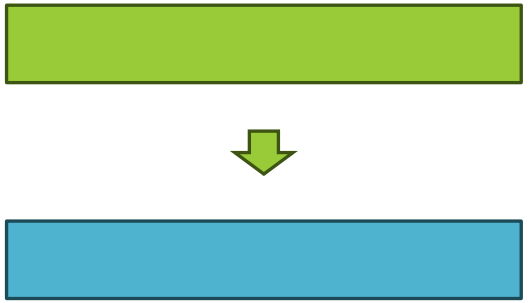
- input size: $2^h \cdot \text{poly}(k)$

Easy to compute, but what about space?

- trivial space: $O(h \cdot \log k)$
- $\log(\textit{input size}) = O(h + \log k)$

Tree Evaluation can be solved in $O((h+\log k)\cdot \text{loglog } k)$ space *[Cook & Mertz 23]*

- so $\text{TreeEval} \in \text{SPACE}(\log n \cdot \text{loglog } n)$
- by catalytic computation, stack can be compressed

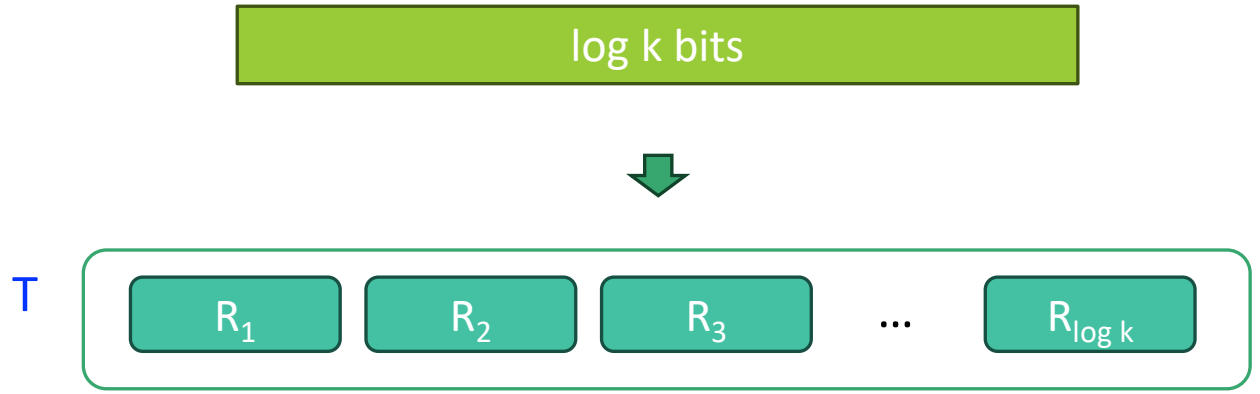


re-use the stack
restore after returned

field $\mathcal{R}$

We work on field $\mathcal{R} = \mathbb{F}_{2^p}$, for p>2·loglog k

- each original value in [k]={1,…,k}, taking log k bits
- extend to log k registers in $\mathcal{R}$
- or seen as one register T in $\mathcal{R}^{\log k}$
- let m=|$\mathcal{R}$|-1=2^p-1



field $\mathcal{R}$

We work on field $\mathcal{R} = \mathbb{F}_{2^p}$, for p>2·loglog k

- each original value in [k]={1,…,k}, taking log k bits
- extend to log k registers in $\mathcal{R}$
- or seen as one register T in $\mathcal{R}^{\log k}$
- let m=$|\mathcal{R}|$-1=2^p-1

Every finite field $\mathcal{R}$ has a primitive root ω of unity of order m=$|\mathcal{R}|$-1

- that is, $\omega^m=1$, and $\sum_{j=1}^{m} \omega^j = 0$
- thus,

$$\sum_{j=1}^{m} \omega^{j \cdot b} = \begin{cases} m & \text{if } b = 0 \\ 0 & \text{if } 0 < b < m \end{cases}$$

Boolean function ⟹ Polynomial

each node has a function f(y,z): $[k] \times [k] \longrightarrow [k]$, so (2·log k)-bit input, (log k)-bit output

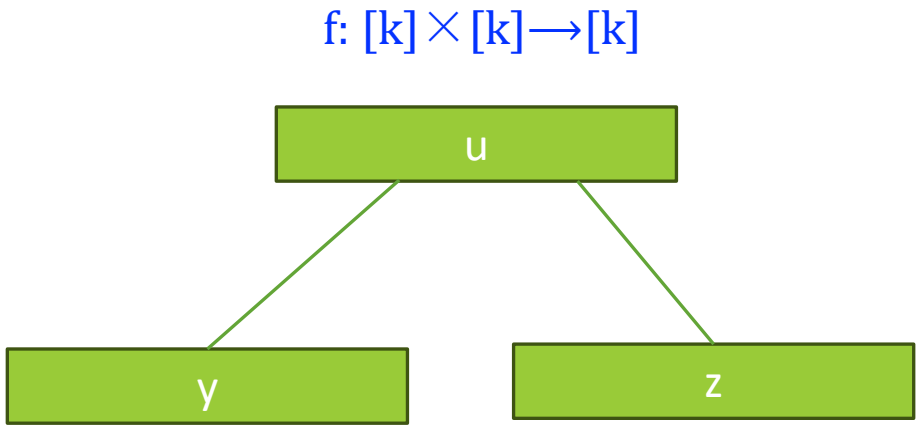
consider *i-th* bit of f, a Boolean function:

$$f_i(y,z) = f_i(y_1,\ldots,y_{\log k},z_1,\ldots,z_{\log k})$$

transfer to a ***multilinear*** polynomial over $\mathcal{R} = \mathbb{F}_{2^p}$:

$$q_i(y,z) = q_i(y_1,\ldots,y_{\log k},z_1,\ldots,z_{\log k})$$

- agree with f_i on {0,1}
- of degree 2·log k

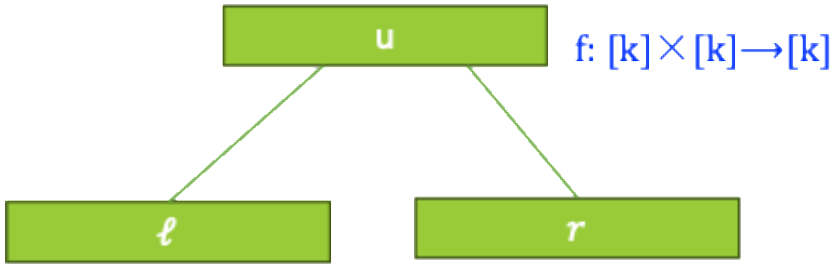


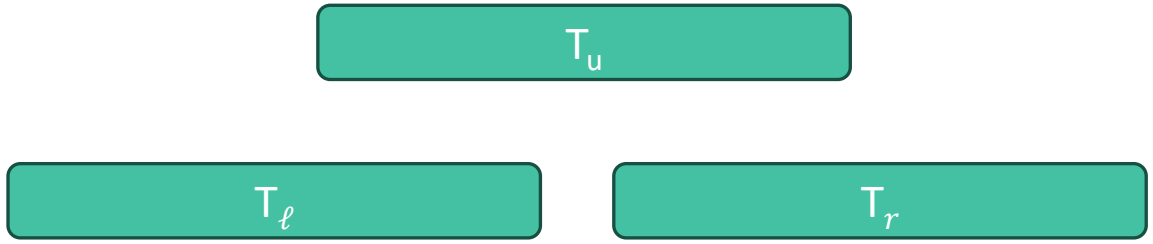
Main theorem

field $\mathcal{R} = \mathbb{F}_{2^p}$, for $p > 2 \cdot \log\log k$, let $m = |\mathcal{R}| - 1 = 2^p - 1$
register T in $\mathcal{R}^{\log k}$, can be seen as log k registers in $\mathcal{R}$

Let P_ℓ, P_r be reversible programs which cleanly compute v_ℓ, v_r into registers T_ℓ, T_r, resp.

Then $f(v_\ell, v_r)$ can be computed cleanly into register T_u





Main theorem

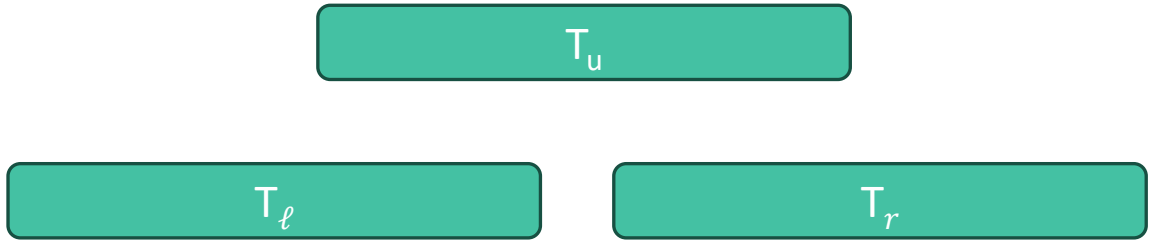
field $\mathcal{R} = \mathbb{F}_{2^p}$, for $p>2\cdot\log\log k$, let $m=|\mathcal{R}|-1=2^p-1$
register T in $\mathcal{R}^{\log k}$, can be seen as log k registers in $\mathcal{R}$

Let P_ℓ, P_r be reversible programs which cleanly compute v_ℓ, v_r into registers T_ℓ, T_r, resp.

Then $f(v_\ell,v_r)$ can be computed cleanly into register T_u

- multilinear polynomial $q_i(y,z) = q_i(y_1,\dots,y_{\log k},z_1,\dots,z_{\log k})$ of $f_i(y,z)$
- $\omega^m=1$ in $\mathcal{R}$

for j ⟵ 1 **to** m **do**
  for i ⟵ 1 **to** log k **do**
    $T_\ell[i] \longleftarrow \omega^j \cdot T_\ell[i]$
    $T_r[i] \longleftarrow \omega^j \cdot T_r[i]$
  P_ℓ, P_r
  for i ⟵ 1 **to** log k **do**
    $T_u[i] \longleftarrow T_u[i] + m^{-1} \cdot q_i(T_\ell,T_r)$
  P_ℓ^{-1}, P_r^{-1}
  for i ⟵ 1 **to** log k **do**
    $T_\ell[i] \longleftarrow \omega^{-j} \cdot T_\ell[i]$
    $T_r[i] \longleftarrow \omega^{-j} \cdot T_r[i]$



Main theorem

field $\mathcal{R} = \mathbb{F}_{2^p}$, for $p>2\cdot\log\log k$, let $m=|\mathcal{R}|-1=2^p-1$
register T in $\mathcal{R}^{\log k}$, can be seen as log k registers in $\mathcal{R}$

Let P_ℓ, P_r be reversible programs which cleanly compute v_ℓ, v_r into registers T_ℓ, T_r, resp.

Then $f(v_\ell,v_r)$ can be computed cleanly into register T_u

- multilinear polynomial $q_i(y,z) = q_i(y_1,\dots,y_{\log k},z_1,\dots,z_{\log k})$ of $f_i(y,z)$
- $\omega^m=1$ in $\mathcal{R}$

for j ⟵ 1 **to** m **do**
  for i ⟵ 1 **to** log k **do**
    $T_\ell[i] \longleftarrow \omega^j \cdot T_\ell[i]$
    $T_r[i] \longleftarrow \omega^j \cdot T_r[i]$
  P_ℓ, P_r
  for i ⟵ 1 **to** log k **do**
    $T_u[i] \longleftarrow T_u[i] + m^{-1} \cdot q_i(T_\ell,T_r)$
  P_ℓ^{-1}, P_r^{-1}
  for i ⟵ 1 **to** log k **do**
    $T_\ell[i] \longleftarrow \omega^{-j} \cdot T_\ell[i]$
    $T_r[i] \longleftarrow \omega^{-j} \cdot T_r[i]$

T_ℓ

$T_\ell[1]$	$T_\ell[2]$	…	$T_\ell[\log k]$

Main theorem

field $\mathcal{R} = \mathbb{F}_{2^p}$, for $p > 2 \cdot \log\log k$, let $m = |\mathcal{R}| - 1 = 2^p - 1$
register T in $\mathcal{R}^{\log k}$, can be seen as log k registers in $\mathcal{R}$

Let P_ℓ, P_r be reversible programs which cleanly compute v_ℓ, v_r into registers T_ℓ, T_r, resp.

Then $f(v_\ell, v_r)$ can be computed cleanly into register T_u

- multilinear polynomial $q_i(y,z) = q_i(y_1,\dots,y_{\log k},z_1,\dots,z_{\log k})$ of $f_i(y,z)$
- $\omega^m = 1$ in $\mathcal{R}$

for j ⟵ 1 **to** m **do**
  for i ⟵ 1 **to** log k **do**
    $T_\ell[i] \longleftarrow \omega^j \cdot T_\ell[i]$
    $T_r[i] \longleftarrow \omega^j \cdot T_r[i]$
  P_ℓ, P_r
  for i ⟵ 1 **to** log k **do**
    $T_u[i] \longleftarrow T_u[i] + m^{-1} \cdot q_i(T_\ell, T_r)$
  P_ℓ^{-1}, P_r^{-1}
  for i ⟵ 1 **to** log k **do**
    $T_\ell[i] \longleftarrow \omega^{-j} \cdot T_\ell[i]$
    $T_r[i] \longleftarrow \omega^{-j} \cdot T_r[i]$

T_ℓ

$\omega^j \cdot T_\ell[1]$	$\omega^j \cdot T_\ell[2]$	…	$\omega^j \cdot T_\ell[\log k]$

Main theorem

field $\mathcal{R} = \mathbb{F}_{2^q}$, for $q > 2 \cdot \log k$, let $m = |\mathcal{R}| - 1 = 2^q - 1$
register T in $\mathcal{R}^{\log k}$, can be seen as log k registers in $\mathcal{R}$

Let P_ℓ, P_r be reversible programs which cleanly compute v_ℓ, v_r into registers T_ℓ, T_r, resp.

Then $f(v_\ell, v_r)$ can be computed cleanly into register T_u

- multilinear polynomial $q_i(y,z) = q_i(y_1,\dots,y_{\log k},z_1,\dots,z_{\log k})$ of $f_i(y,z)$
- $\omega^m = 1$ in $\mathcal{R}$

for j ⟵ 1 **to** m **do**
  for i ⟵ 1 **to** log k **do**
    $T_\ell[i] \longleftarrow \omega^j \cdot T_\ell[i]$
    $T_r[i] \longleftarrow \omega^j \cdot T_r[i]$
  P_ℓ, P_r
  for i ⟵ 1 **to** log k **do**
    $T_u[i] \longleftarrow T_u[i] + m^{-1} \cdot q_i(T_\ell, T_r)$
  P_ℓ^{-1}, P_r^{-1}
  for i ⟵ 1 **to** log k **do**
    $T_\ell[i] \longleftarrow \omega^{-j} \cdot T_\ell[i]$
    $T_r[i] \longleftarrow \omega^{-j} \cdot T_r[i]$

T_ℓ

$\omega^j \cdot T_\ell[1] + v_\ell[1]$	$\omega^j \cdot T_\ell[2] + v_\ell[2]$	…	$\omega^j \cdot T_\ell[\log k] + v_\ell[\log k]$

Main theorem

field $\mathcal{R} = \mathbb{F}_{2^p}$, for p>2·loglog k, let m=|$\mathcal{R}$|-1=2^p-1
register T in $\mathcal{R}^{\log k}$, can be seen as log k registers in $\mathcal{R}$

Let P_ℓ, P_r be reversible programs which cleanly compute v_ℓ, v_r into registers T_ℓ, T_r, resp.

Then $f(v_\ell, v_r)$ can be computed cleanly into register T_u

- multilinear polynomial $q_i(y,z) = q_i(y_1,\dots,y_{\log k},z_1,\dots,z_{\log k})$ of $f_i(y,z)$
- $\omega^m=1$ in $\mathcal{R}$

for j ⟵ 1 **to** m **do**
  for i ⟵ 1 **to** log k **do**
    $T_\ell[i] \longleftarrow \omega^j \cdot T_\ell[i]$
    $T_r[i] \longleftarrow \omega^j \cdot T_r[i]$
  P_ℓ, P_r
  for i ⟵ 1 **to** log k **do**
    $T_u[i] \longleftarrow T_u[i] + m^{-1} \cdot q_i(T_\ell, T_r)$
  P_ℓ^{-1}, P_r^{-1}
  for i ⟵ 1 **to** log k **do**
    $T_\ell[i] \longleftarrow \omega^{-j} \cdot T_\ell[i]$
    $T_r[i] \longleftarrow \omega^{-j} \cdot T_r[i]$

T_ℓ

$\omega^j \cdot T_\ell[1]+v_\ell[1]$	$\omega^j \cdot T_\ell[2]+v_\ell[2]$	…	$\omega^j \cdot T_\ell[\log k]+v_\ell[\log k]$

$$T_u[i] \longleftarrow T_u[i] + m^{-1} \cdot q_i(\omega^j \cdot T_\ell + v_\ell,\ \omega^j \cdot T_r + v_r)$$

Main theorem

field $\mathcal{R} = \mathbb{F}_{2^p}$, for p>2·loglog k, let m=|$\mathcal{R}$|-1=2^p-1
register T in $\mathcal{R}^{\log k}$, can be seen as log k registers in $\mathcal{R}$

Let P_ℓ, P_r be reversible programs which cleanly compute v_ℓ, v_r in

degree of $q_i(\cdot)$ is $2\cdot\log k < m$

Then $f(v_\ell,v_r)$ can be computed cleanly into register T_u

- multilinear polynomial $q_i(y,z) = q_i(y_1,\dots,y_{\log k},z_1,\dots,z_{\log k})$ of $f_i(y,z)$
- $\omega^m=1$ in $\mathcal{R}$

for j ⟵ 1 **to** m **do**
  for i ⟵ 1 **to** log k **do**
    $T_\ell[i] \longleftarrow \omega^j \cdot T_\ell[i]$
    $T_r[i] \longleftarrow \omega^j \cdot T_r[i]$
  P_ℓ, P_r
  for i ⟵ 1 **to** log k **do**
    $T_u[i] \longleftarrow T_u[i] + m^{-1} \cdot q_i(T_\ell,T_r)$
  P_ℓ^{-1}, P_r^{-1}
  for i ⟵ 1 **to** log k **do**
    $T_\ell[i] \longleftarrow \omega^{-j} \cdot T_\ell[i]$
    $T_r[i] \longleftarrow \omega^{-j} \cdot T_r[i]$

T_ℓ

$\omega^j \cdot T_\ell[1]+v_\ell[1]$	$\omega^j \cdot T_\ell[2]+v_\ell[2]$	…	$\omega^j \cdot T_\ell[\log k]+v_\ell[\log k]$

$$T_u[i] \longleftarrow T_u[i] + m^{-1} \cdot q_i(\omega^j \cdot T_\ell+v_\ell,\ \omega^j \cdot T_r+v_r)$$

sum over j=1,…,m :

$$T_u[i] \longleftarrow T_u[i] + m^{-1} \cdot \sum_{j=1}^{m} q_i(\omega^j \cdot T_\ell+v_\ell,\ \omega^j \cdot T_r+v_r)$$

Main theorem

field $\mathcal{R} = \mathbb{F}_{2^p}$, for p>2·loglog k, let m=|$\mathcal{R}$|-1=2^p-1
register T in $\mathcal{R}^{\log k}$, can be seen as log k registers in $\mathcal{R}$

Let P_ℓ, P_r be reversible programs which cleanly compute v_ℓ, v_r in

degree of $q_i(\cdot)$ is $2\cdot\log k < m$

Then $f(v_\ell,v_r)$ can be computed cleanly into register T_u

- multilinear polynomial $q_i(y,z) = q_i(y_1,\ldots,y_{\log k},z_1,\ldots,z_{\log k})$ of $f_i(y,z)$
- $\omega^m=1$ in $\mathcal{R}$

for j ⟵ 1 **to** m **do**
  for i ⟵ 1 **to** log k **do**
    $T_\ell[i] \leftarrow \omega^j \cdot T_\ell[i]$
    $T_r[i] \leftarrow \omega^j \cdot T_r[i]$
  P_ℓ, P_r
  for i ⟵ 1 **to** log k **do**
    $T_u[i] \leftarrow T_u[i] + m^{-1} \cdot q_i(T_\ell,T_r)$
  P_ℓ^{-1}, P_r^{-1}
  for i ⟵ 1 **to** log k **do**
    $T_\ell[i] \leftarrow \omega^{-j} \cdot T_\ell[i]$
    $T_r[i] \leftarrow \omega^{-j} \cdot T_r[i]$

$$T_u[i] \leftarrow T_u[i] + m^{-1} \cdot \sum_{j=1}^{m} q_i(\omega^j \cdot T_\ell + v_\ell,\ \omega^j \cdot T_r + v_r)$$

- since $\sum_{j=1}^{m} \omega^{j\cdot b} = \begin{cases} m & \text{if } b = 0 \\ 0 & \text{if } 0 < b < m \end{cases}$
- all term with ω^j are cancelled

Thus, $T_u[i] \leftarrow T_u[i] + q_i(v_\ell, v_r)$

Main theorem

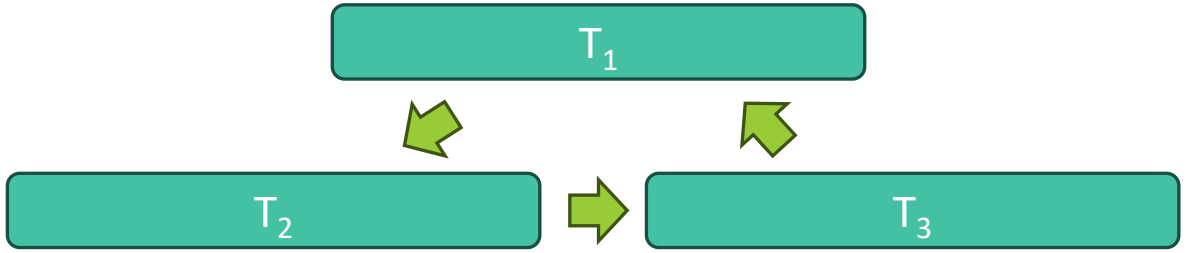
field $\mathcal{R} = \mathbb{F}_{2^p}$, for $p>2\cdot\log\log k$, let $m=|\mathcal{R}|-1=2^p-1$
register T in $\mathcal{R}^{\log k}$, can be seen as log k registers in $\mathcal{R}$

Let P_ℓ, P_r be reversible programs which cleanly compute v_ℓ, v_r into registers T_ℓ, T_r, resp.

Then $f(v_\ell,v_r)$ can be computed cleanly into register T_u

- multilinear polynomial $q_i(y,z) = q_i(y_1,\dots,y_{\log k},z_1,\dots,z_{\log k})$ of $f_i(y,z)$
- $\omega^m=1$ in $\mathcal{R}$

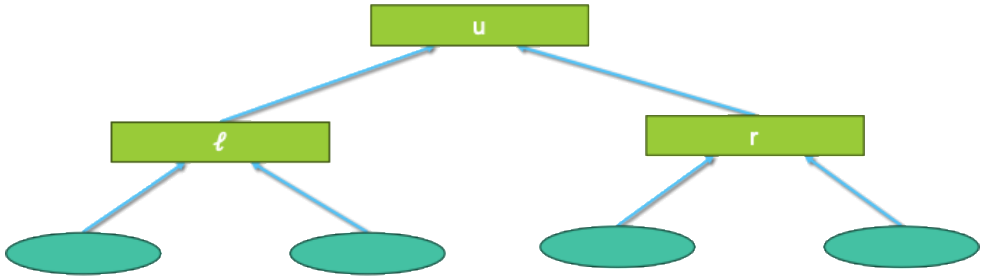
for $j \leftarrow 1$ **to** m **do**
  for $i \leftarrow 1$ **to** log k **do**
    $T_\ell[i] \leftarrow \omega^j \cdot T_\ell[i]$
    $T_r[i] \leftarrow \omega^j \cdot T_r[i]$
  P_ℓ, P_r
  for $i \leftarrow 1$ **to** log k **do**
    $T_u[i] \leftarrow T_u[i] + m^{-1} \cdot q_i(T_\ell,T_r)$
  P_ℓ^{-1}, P_r^{-1}
  for $i \leftarrow 1$ **to** log k **do**
    $T_\ell[i] \leftarrow \omega^{-j} \cdot T_\ell[i]$
    $T_r[i] \leftarrow \omega^{-j} \cdot T_r[i]$



just need 3 registers T_1, T_2, T_3
space: $O(\log|\mathcal{R}|\cdot\log k)=O(p\cdot\log k)=O(\log k\cdot\log\log k)$

length of program: $\sim(4m)^h$
(h: height of tee)
so we need $O(h\cdot\log m)=O(h\cdot\log\log k)$ space for pointer of the program

Tree Evaluation



Complete binary tree

- input size: $2^h \cdot \text{poly}(k)$
- trivial space: $O(h \cdot \log k)$
- $\log(\textit{input size}) = O(h + \log k)$

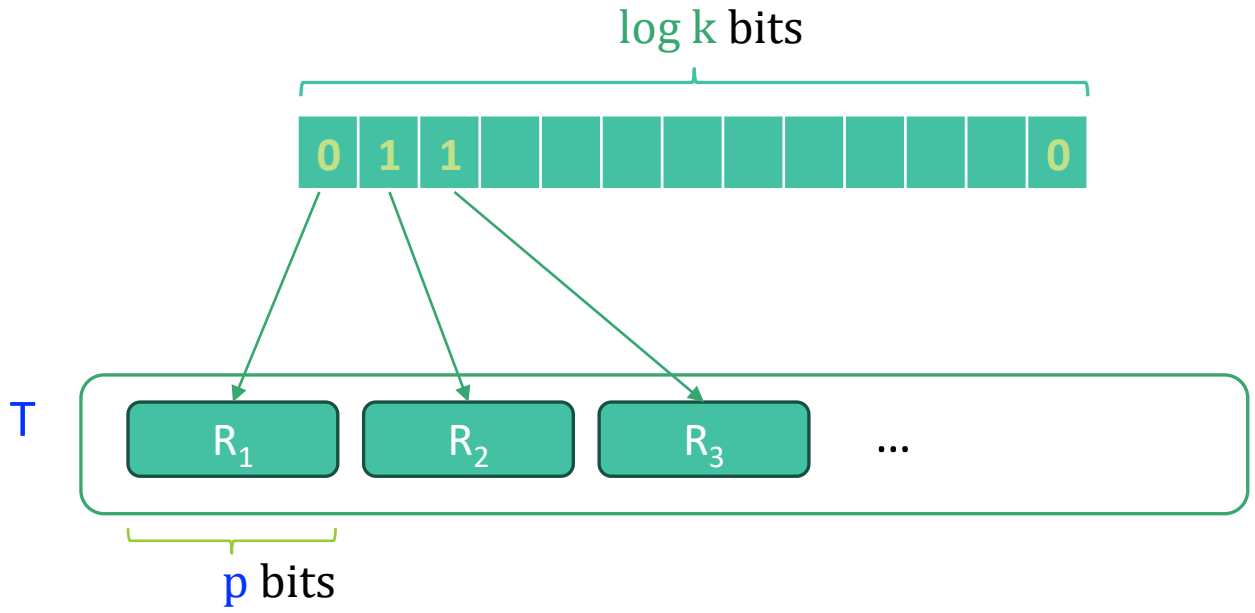
Tree Evaluation can be solved in $O((h+\log k)\cdot \text{loglog } k)$ space *[Cook & Mertz 23]*

- so TreeEval $\in$ SPACE(log n·loglog n)
- by catalytic computation, stack can be compressed
- *can be improved to $O(h \cdot \text{loglog } k + \log k)$ space*

Improvement

Now every bit takes one register

log k bits extend to log k·loglog k bits



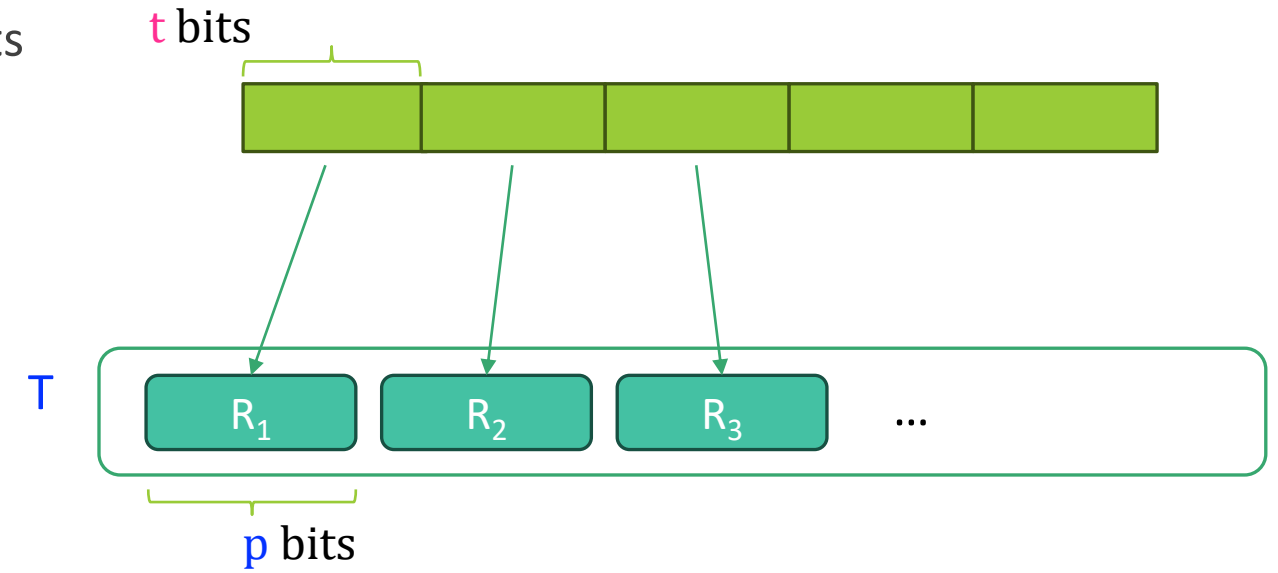
Improvement

original log k bits divided into blocks of t bits

- a=log k/t blocks
- make t=loglog k

function f: [k]×[k]⟶[k] becomes

- input: $2a$ values in $\mathbb{F}_{2^t}$
- output: a values in $\mathbb{F}_{2^t}$
- every value in $\mathbb{F}_{2^t}$ use a p-bit register



function $\Longrightarrow$ polynomial

For a function $g(x_1,\ldots,x_n)\colon \mathbb{F}^n \longrightarrow \mathbb{F}$ where $\mathbb{F}$ is a finite field, write $g(\cdot)$ as

$$g(x_1,\ldots,x_n) = \sum_{(a_1,\ldots,a_n)\in\mathbb{F}^n} g(a_1,\ldots,a_n) \prod_{i=1}^{n} [\![x_i = a_i]\!]$$

- again using Fermat's little theorem:

$$[\![x_i = a_i]\!] = 1 - (x_i - a_i)^{|\mathbb{F}|-1}$$

- so $g(x_1,\ldots,x_n)\colon \mathbb{F}^n \longrightarrow \mathbb{F}$ change to a polynomial of degree $n(|\mathbb{F}| - 1)$

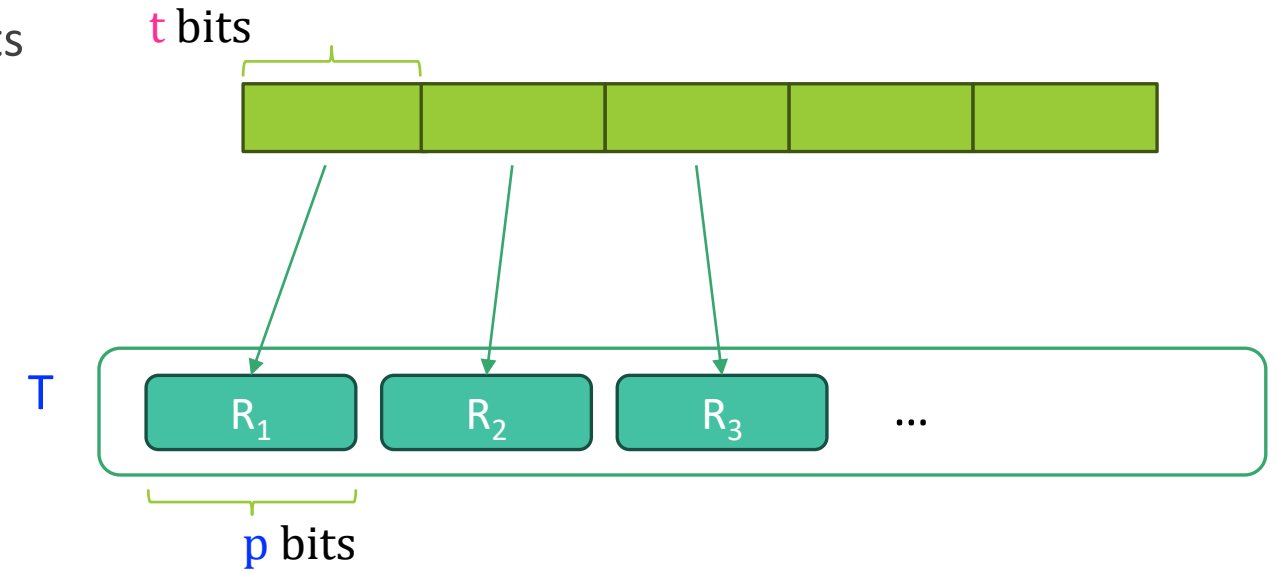
Improvement

original log k bits divided into blocks of t bits

- a=log k/t blocks
- make t=loglog k

function f: [k]×[k]⟶[k] becomes

- input: $2a$ values in $\mathbb{F}_{2^t}$
- output: a values in $\mathbb{F}_{2^t}$
- every value in $\mathbb{F}_{2^t}$ use a p-bit register



transfer each f_i to a polynomial of degree $<2a\cdot 2^t<\log^2 k$

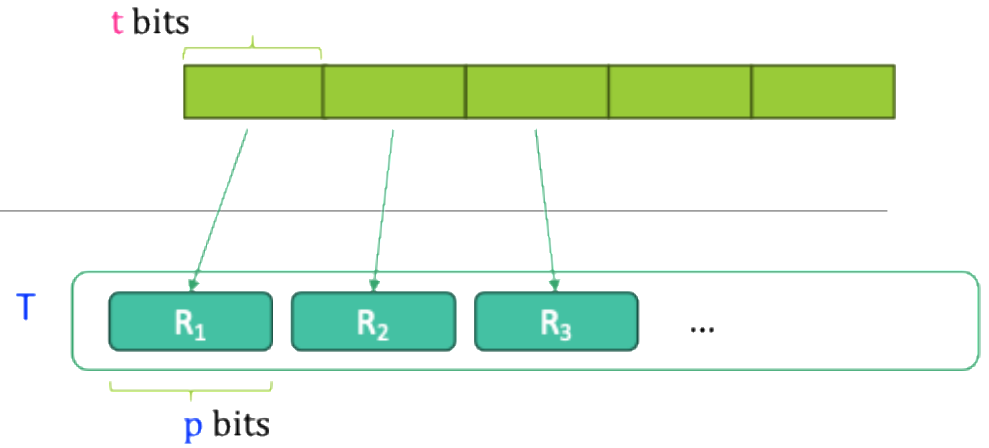
- let p=2·loglog k, then the degree $< 2^p$

Improvement

original log k bits divided into blocks of t bits
- a=log k/t blocks
- make t=loglog k

function f: [k]×[k]⟶[k] becomes
- input: $2a$ values in $\mathbb{F}_{2^t}$
- output: a values in $\mathbb{F}_{2^t}$
- every value in $\mathbb{F}_{2^t}$ use a p-bit register

transfer each f_i to a polynomial of degree $<2a\cdot 2^t<\log^2 k$
- let p=2·loglog k, then the degree $< 2^p$
- $m=2^p-1$, you can check the algorithm works
- space for registers: O(p·log k/t)=O(log k)



for j ⟵ 1 **to** m **do**
  for i ⟵ 1 **to** log k **do**
    $T_\ell[i] \longleftarrow \omega^j \cdot T_\ell[i]$
    $T_r[i] \longleftarrow \omega^j \cdot T_r[i]$
  P_ℓ, P_r
  for i ⟵ 1 **to** log k **do**
    $T_u[i] \longleftarrow T_u[i] + m^{-1} \cdot q_i(T_\ell, T_r)$
  P_ℓ^{-1}, P_r^{-1}
  for i ⟵ 1 **to** log k **do**
    $T_\ell[i] \longleftarrow \omega^{-j} \cdot T_\ell[i]$
    $T_r[i] \longleftarrow \omega^{-j} \cdot T_r[i]$

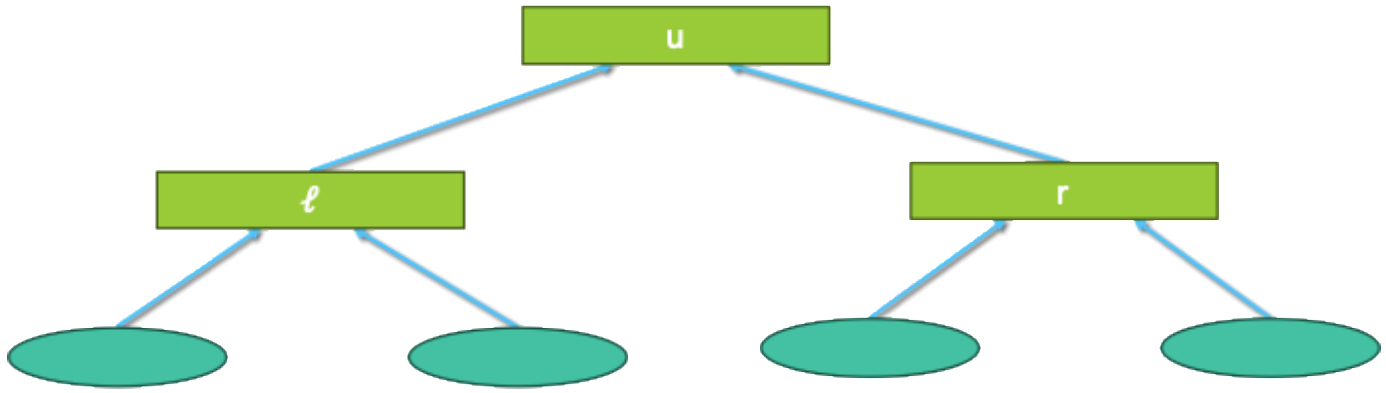
Tree Evaluation

Complete binary tree

- input size: $2^h \cdot \text{poly}(k)$
- trivial space: $O(h \cdot \log k)$

Tree Evaluation can be solved in $O(h \cdot \text{loglog } k + \log k)$ space *[Cook & Mertz 23]*

- so $\text{TreeEval} \in \text{SPACE}(\log n \cdot \text{loglog } n)$



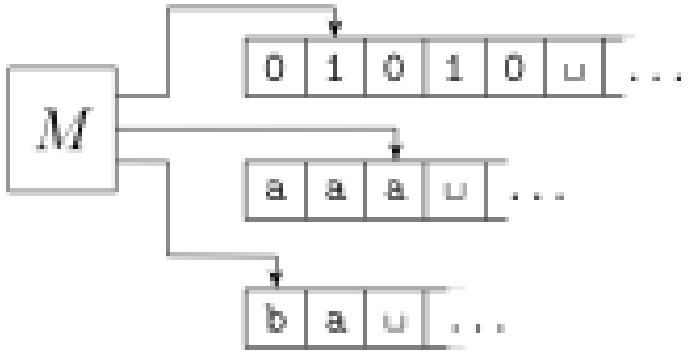
Simulating Time With Square-Root Space

(If TIME[t(n)] and SPACE[s(n)] are defined on multitape Turing machines)

For t(n)≥n,

$$\mathrm{TIME}[t(n)] \subseteq \mathrm{SPACE}\left[\sqrt{t(n) \cdot \log t(n)}\right]$$

- [Williams *STOC 2025 best paper award*]



Simulating Time With Square-Root Space

(If TIME[t(n)] and SPACE[s(n)] are defined on multitape Turing machines)

For t(n)≥n,

$$\mathrm{TIME}[t(n)] \subseteq \mathrm{SPACE}\left[\sqrt{t(n) \cdot \log t(n)}\right]$$

- [Williams *STOC 2025 best paper award*]

Previous results:

- $\mathrm{TIME}[t(n)] \subseteq \mathrm{SPACE}[t(n)/\log t(n)]$ *[Hopcroft, Paul, and Valiant 1975]*
- one tape TM in time $t(n)$ can be simulated in space $O\left(\sqrt{t(n)}\right)$ *[Hopcroft and Ullman 1968]*

Simulating Time With Square-Root Space

(If TIME[t(n)] and SPACE[s(n)] are defined on multitape Turing machines)

For t(n)≥n,

$$\mathrm{TIME}[t(n)] \subseteq \mathrm{SPACE}\left[\sqrt{t(n) \cdot \log t(n)}\right]$$

- [Williams *STOC 2025 best paper award*]

- by reduction to Tree Evaluation problem
- today we look at a basic case: for t(n)≥n^2, $\mathrm{TIME}[t(n)] \subseteq \mathrm{SPACE}\left[\sqrt{t(n)} \cdot \log t(n)\right]$

Oblivious Turing machines

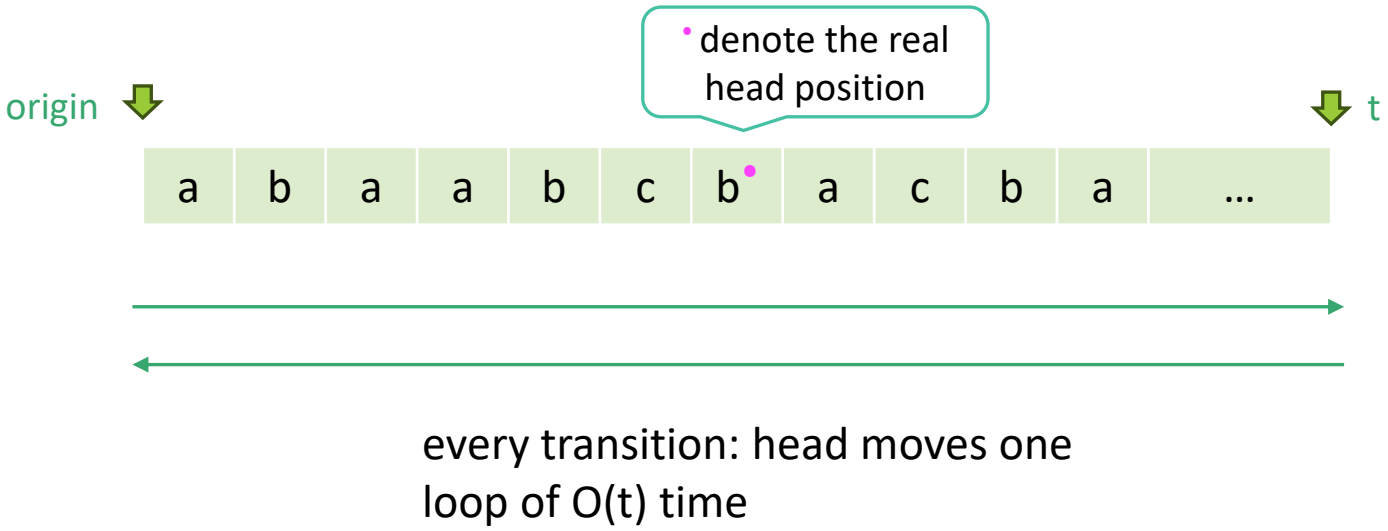
Turing machine M is *oblivious* if for every input x of length $n=|x|$, the tape head movements of M depend only on n, and not on x itself

THM.*[Pippenger & Fischer 1979]* Every multitape TM in time t(n) is equivalent to a two-tape oblivious TM in time $T(n)=O(t(n)\cdot \log t(n))$

Oblivious Turing machines

Turing machine M is *oblivious* if for every input x of length n=|x|, the tape head movements of M depend only on n, and not on x itself

Easy: time t machine $\Longrightarrow$ time $O(t^2)$ oblivious machine

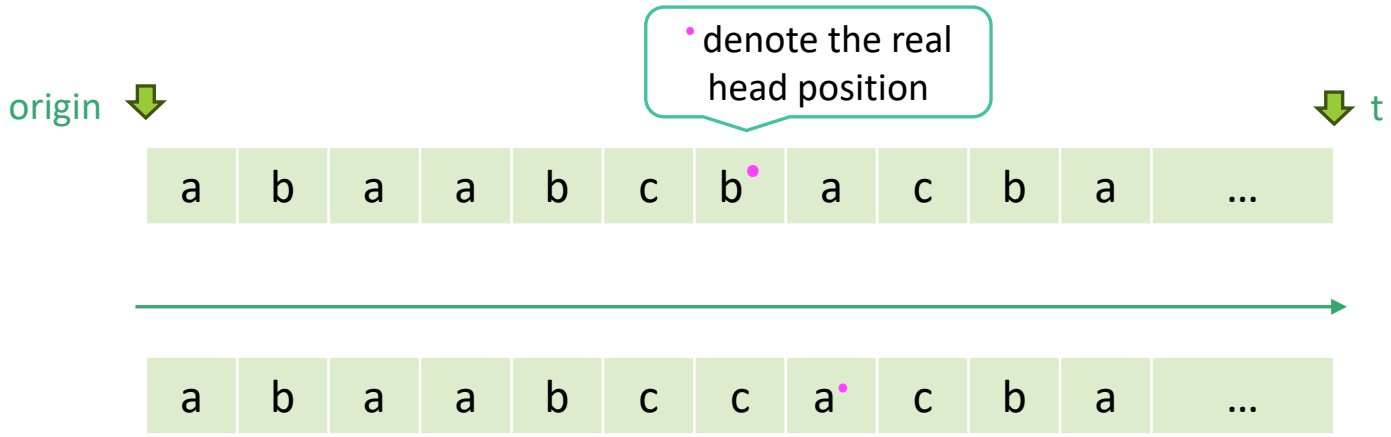


every transition: head moves one
loop of O(t) time

Oblivious Turing machines

Turing machine M is *oblivious* if for every input x of length n=|x|, the tape head movements of M depend only on n, and not on x itself

Easy: time t machine $\Longrightarrow$ time $O(t^2)$ oblivious machine

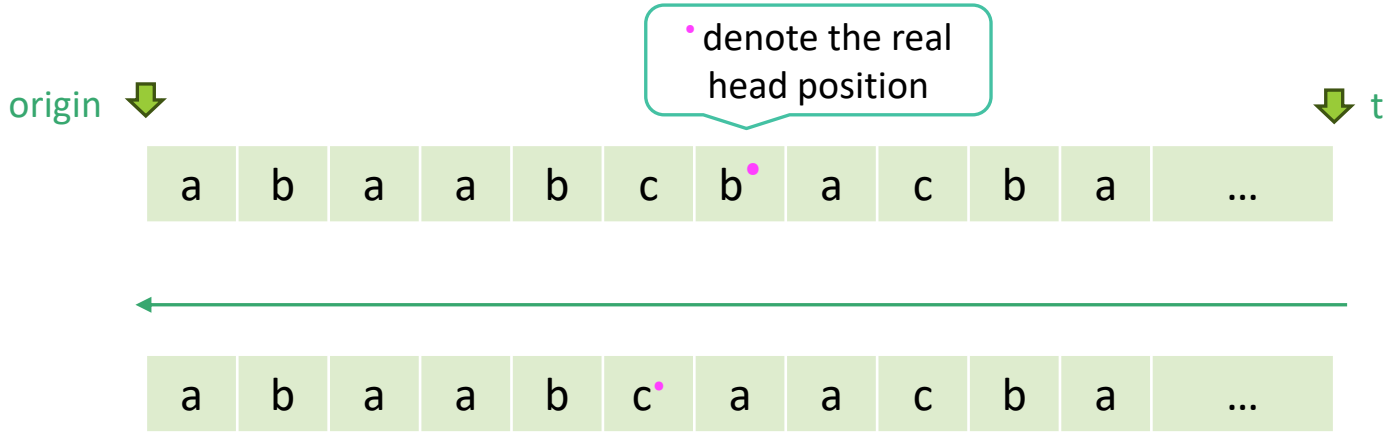


Oblivious Turing machines

Turing machine M is *oblivious* if for every input x of length n=|x|, the tape head movements of M depend only on n, and not on x itself

Easy: time t machine ⟹ time $O(t^2)$ oblivious machine

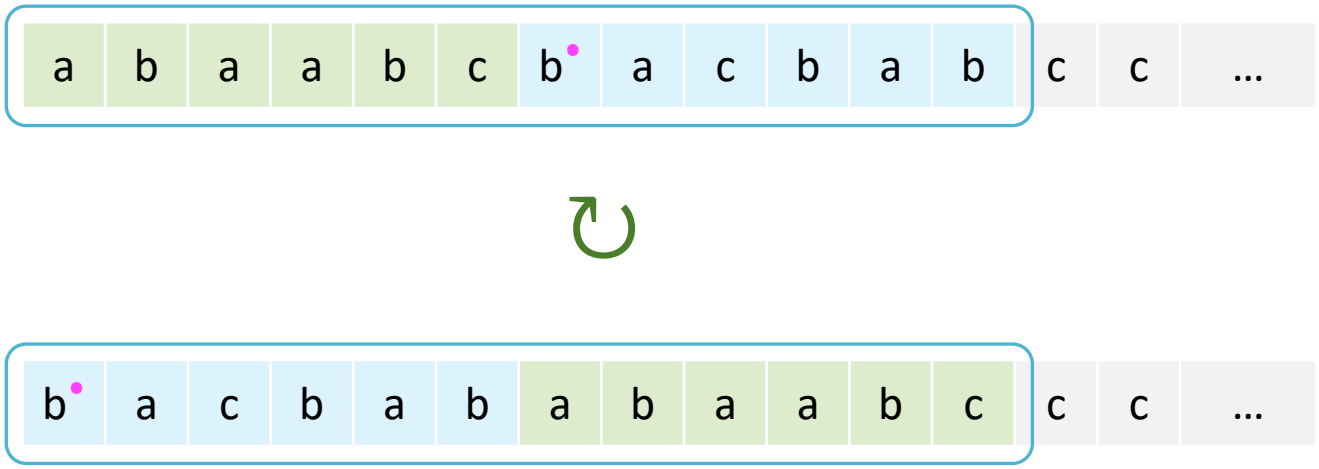
- multitape TM can be simulated similarly



Oblivious Turing machines

time t machine ⟹ time O(t·log t) oblivious machine

- rotate the real head position close to origin

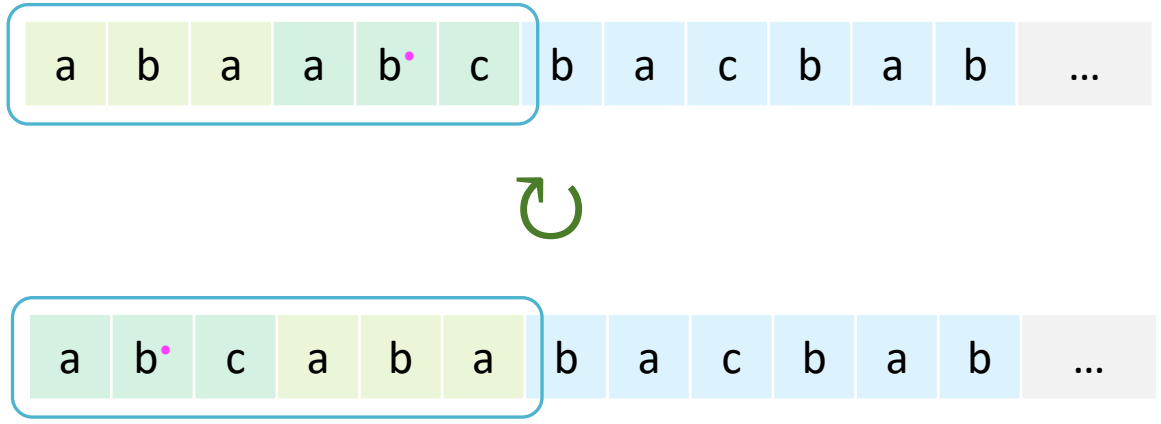


Oblivious Turing machines

time t machine ⟹ time O(t·log t) oblivious machine

- rotate the real head position close to origin
- P(H): position of head on the tape

```
//currently P(H)≤T
//perform T steps, so P(H)≤2T in the end
PERFORM(T) {
   if (T==1), execute directly
   if (P(H)>T/2)
       rotate first T cells by T/2 so that P(H)≤T/2
   PERFORM(T/2)
   if (P(H)>T/2)
       rotate first T cells by T/2 so that P(H)≤T/2
   PERFORM(T/2)
   rotate back    //use another tape to keep track of
}
```



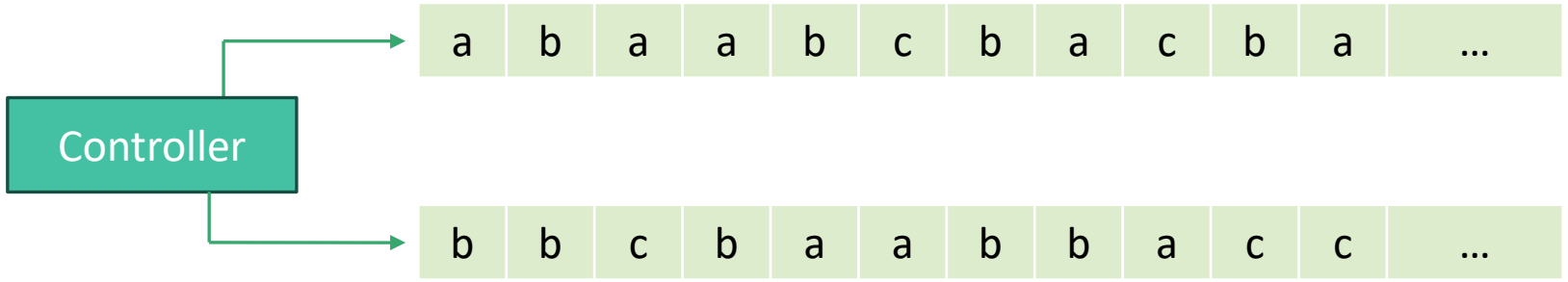
Time(T)=2Time(T/2)+O(T)
so Time(T)=O(T·log T)

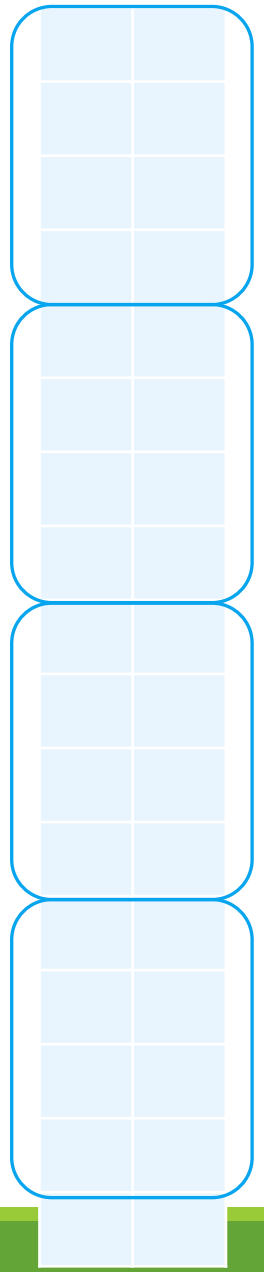
Tape/time blocks

M: Time T oblivious two-tape TM with $T \geq n^2$

Divide both time and tapes into blocks of length L

- *#blocks* in time and tape: B=O(T/L)
- L is about $\sqrt{T \cdot \log T}$, so L≥n



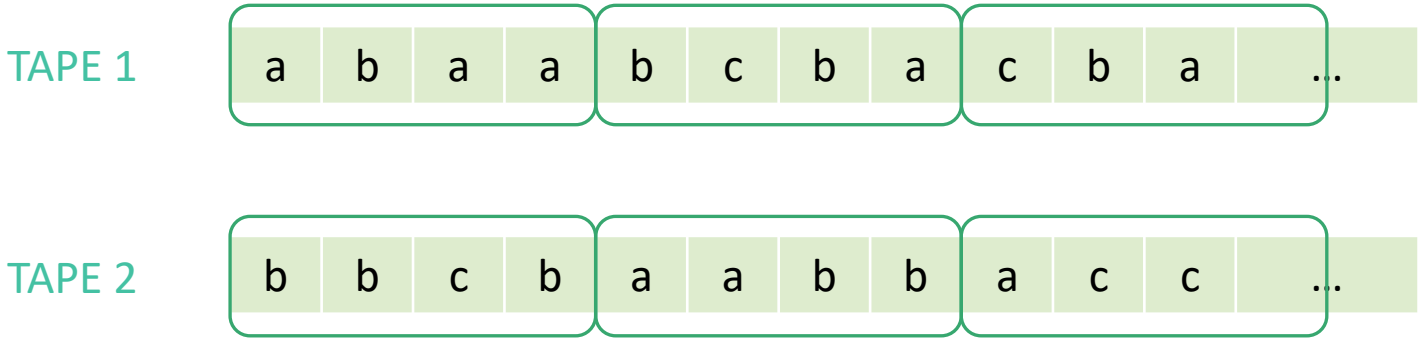


M: Time T oblivious two-tape TM with $T \geq n^2$

Divide both time and tapes into blocks of length $L \geq n$

- *#blocks* in time and tape: $B = O(T/L)$

In time t, at which tapes blocks the head positions are?



TIME

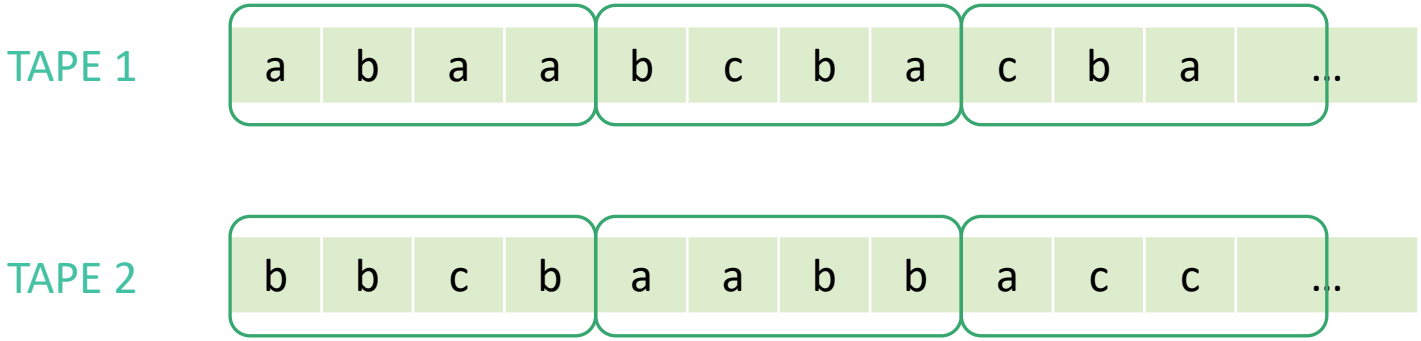
2	3
2	3
2	2
3	2
3	2
2	2
2	1
3	1
3	2
3	2
3	1
4	2
4	2
4	2
3	2
3	3
4	3

M: Time T oblivious two-tape TM with $T \geq n^2$

Divide both time and tapes into blocks of length $L \geq n$

- *#blocks* in time and tape: $B=O(T/L)$

In time t, at which tapes blocks the head positions are?



TIME

2	3
2	3
2	2
3	2
3	2
2	2
2	1
3	1
3	2
3	2
3	1
4	2
4	2
4	2
3	2
3	3
4	3

M: Time T oblivious two-tape TM with $T \geq n^2$

Divide both time and tapes into blocks of length $L \geq n$

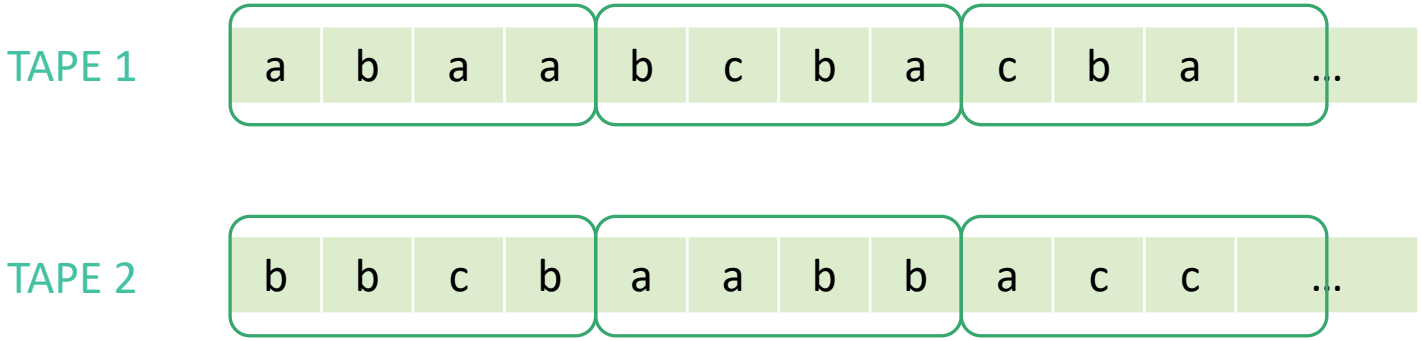
- *#blocks* in time and tape: $B=O(T/L)$

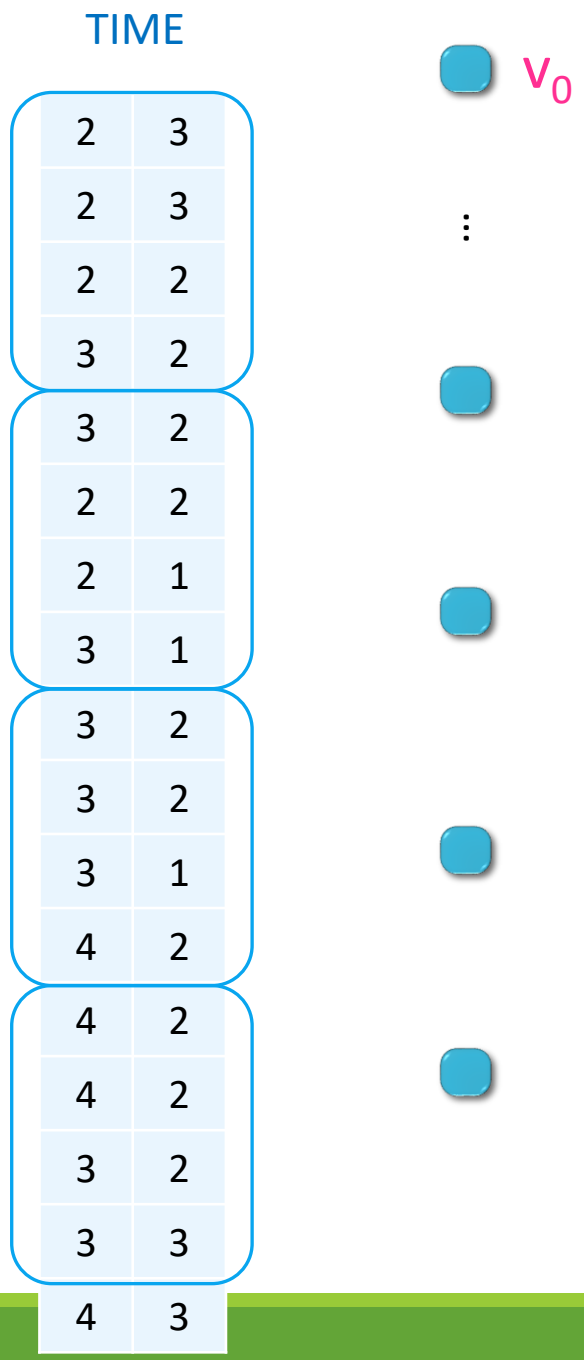
In time t, at which tapes blocks the head positions are?

Every time block j can only contain 2 tapes blocks in each tape

Construct a directed acyclic graph:

- a node v_i for each time block i, with an extra v_0





M: Time T oblivious two-tape TM with $T \geq n^2$

Divide both time and tapes into blocks of length $L \geq n$

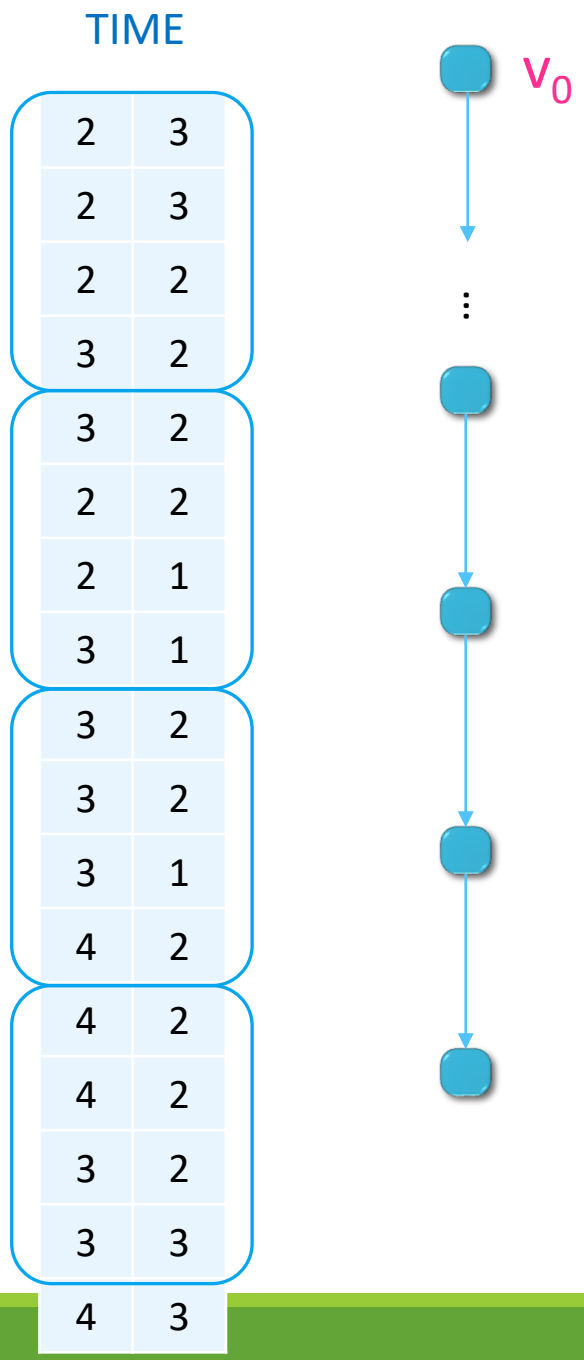
- *#blocks* in time and tape: $B = O(T/L)$

In time t, at which tapes blocks the head positions are?

Every time block j can only contain 2 tapes blocks in each tape

Construct a directed acyclic graph:

- a node v_i for each time block i, with an extra v_0



M: Time T oblivious two-tape TM with $T \geq n^2$

Divide both time and tapes into blocks of length $L \geq n$

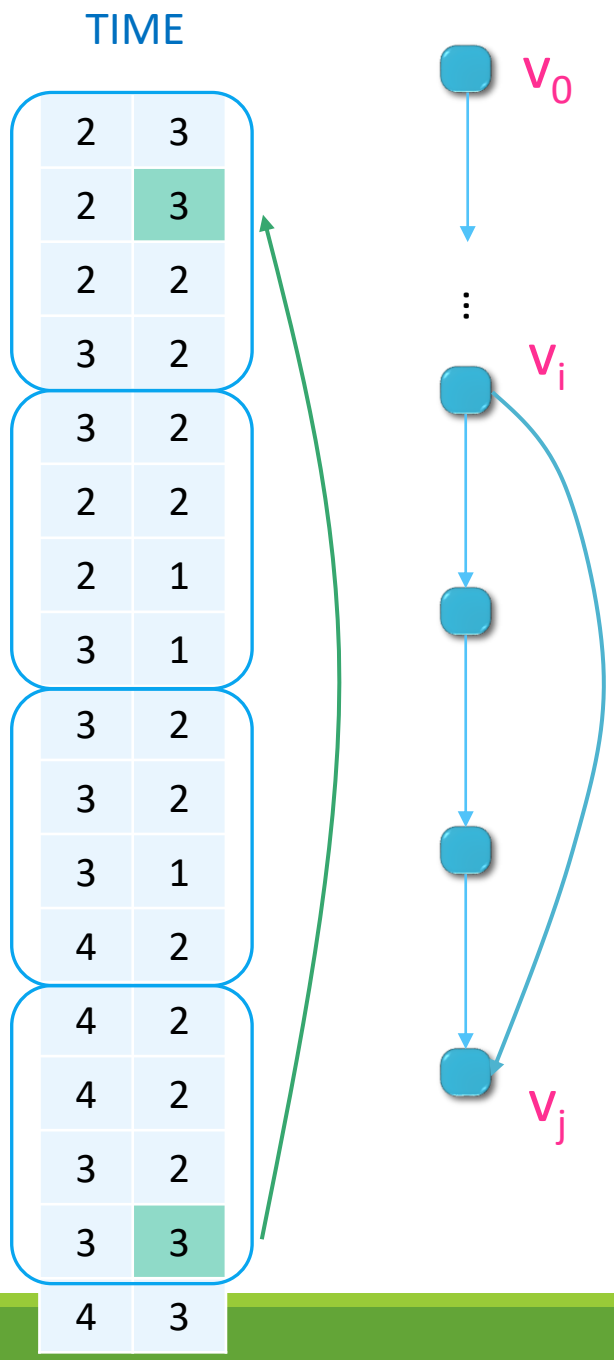
- *#blocks* in time and tape: $B=O(T/L)$

In time t, at which tapes blocks the head positions are?

Every time block j can only contain 2 tapes blocks in each tape

Construct a directed acyclic graph:

- a node v_i for each time block i, with an extra v_0
- add edge (v_i, v_{i+1})
- if time block j contains a tape block C, and last time C being visited in is time block i, add an edge (v_i, v_j)



M: Time T oblivious two-tape TM with $T \geq n^2$

Divide both time and tapes into blocks of length $L \geq n$

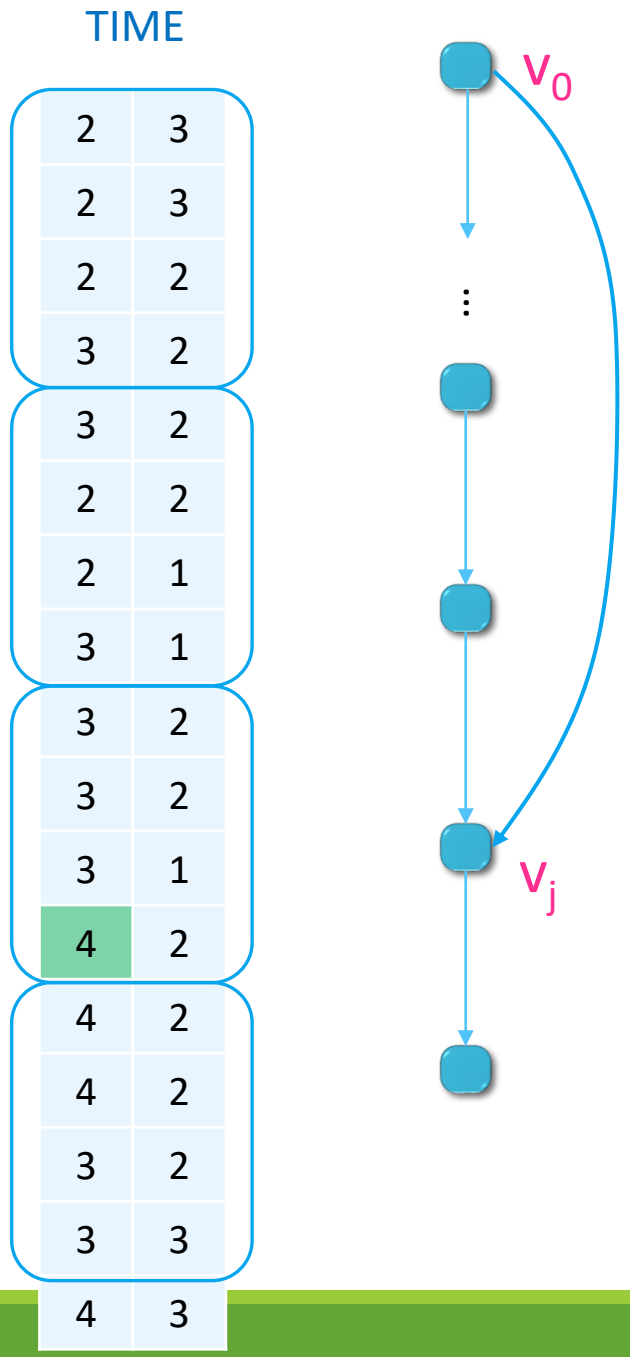
- *#blocks* in time and tape: $B=O(T/L)$

In time t, at which tapes blocks the head positions are?

Every time block j can only contain 2 tapes blocks in each tape

Construct a directed acyclic graph:

- a node v_i for each time block i, with an extra v_0
- add edge (v_i, v_{i+1})
- if time block j contains a tape block C, and last time C being visited in is time block i, add an edge (v_i, v_j)



M: Time T oblivious two-tape TM with $T \geq n^2$

Divide both time and tapes into blocks of length $L \geq n$

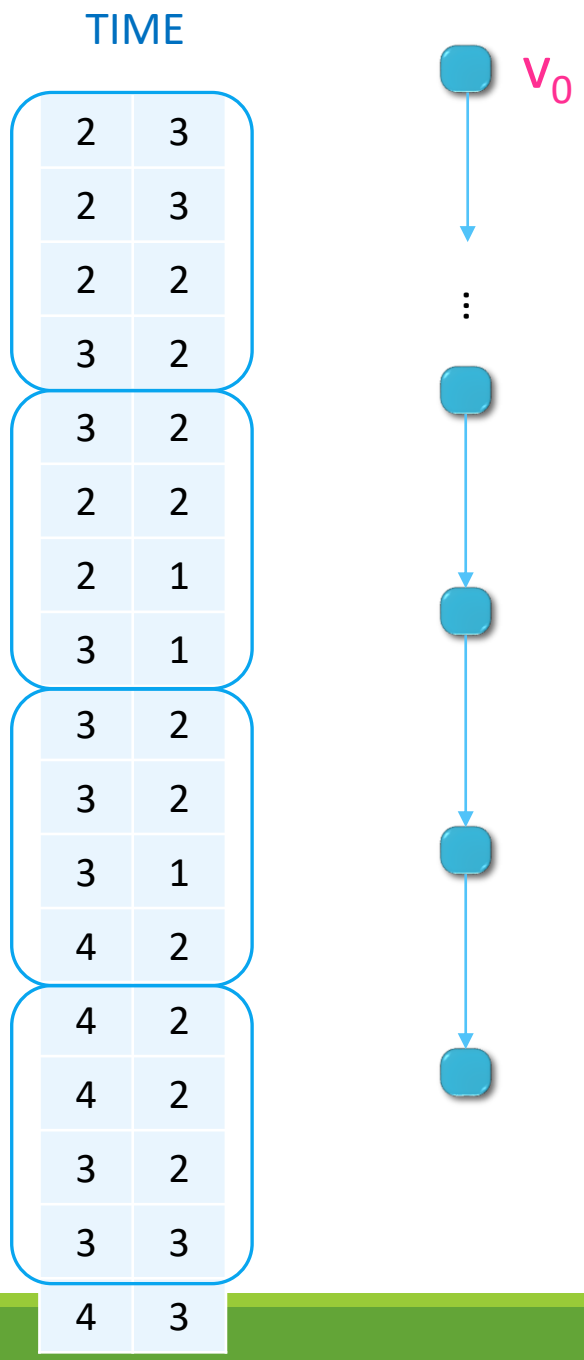
- *#blocks* in time and tape: $B=O(T/L)$

In time t, at which tapes blocks the head positions are?

Every time block j can only contain 2 tapes blocks in each tape

Construct a directed acyclic graph:

- a node v_i for each time block i, with an extra v_0
- add edge (v_i, v_{i+1})
- if time block j contains a tape block C, and last time C being visited in is time block i, add an edge (v_i, v_j)
- if time block j contains a tape block C, but C never visited before, add an edge (v_0, v_j)



M: Time T oblivious two-tape TM with $T \geq n^2$

Divide both time and tapes into blocks of length $L \geq n$

- *#blocks* in time and tape: $B = O(T/L)$

In time t, at which tapes blocks the head positions are?

Every time block j can only contain 2 tapes blocks in each tape

Construct a directed acyclic graph:

- a node v_i for each time block i, with an extra v_0
- add edge (v_i, v_{i+1})
- if time block j contains a tape block C, and last time C being visited in is time block i, add an edge (v_i, v_j)
- if time block j contains a tape block C, but C never visited before, add an edge (v_0, v_j)

Each node has ≤5 predecessors

- its content only depend those predecessors
- the graph is logspace uniform

Tree Evaluation

M: Time T oblivious two-tape TM with $T \geq n^2$
Divide both time and tapes into blocks of length $L \geq n$
#blocks in time and tape: $B = O(T/L)$

use Tree Evaluation to compute the nodes in the graph

- binary tree $\Longrightarrow$ d-ary tree for a constant $d=5$
- height $h=O(B)$
- alphabet $[k]$, $k=2^{O(L)}$

Tree Evaluation can be solved in $O(h \cdot \log\log k + \log k)$ space *[Cook & Mertz 23]*

- it is $O(L+B \cdot \log L)$ space
- let $L \leftarrow \sqrt{T \cdot \log T}$, it is $O\left(\sqrt{T \cdot \log T}\right)$ space

Tree Evaluation

use Tree Evaluation to compute the nodes in the graph

- binary tree $\Longrightarrow$ d-ary tree for a constant d=5
- height h=O(B)
- alphabet [k], $k=2^{O(L)}$

Tree Evaluation can be solved in O(h·loglog k+log k) space *[Cook & Mertz 23]*

- it is O(L+B·log L) space
- let $L \leftarrow \sqrt{T \cdot \log T}$, it is $O\left(\sqrt{T \cdot \log T}\right)$ space

Every multitape TM in time t(n) is equivalent to a two-tape oblivious TM in time T(n)=O(t(n)·log t(n))

- it is $O\left(\sqrt{t(n)} \log t(n)\right)$ space

Simulating Time With Square-Root Space

(If TIME[t(n)] and SPACE[s(n)] are defined on multitape Turing machines)

We have seen: for $t(n) \geq n^2$,

$$\mathrm{TIME}[t(n)] \subseteq \mathrm{SPACE}\left[\sqrt{t(n)} \cdot \log t(n)\right]$$

Further improved to: for $t(n) \geq n$, [Williams *STOC 2025*]

$$\mathrm{TIME}[t(n)] \subseteq \mathrm{SPACE}\left[\sqrt{t(n) \cdot \log t(n)}\right]$$

Simulating Time With Square-Root Space

(If TIME[t(n)] and SPACE[s(n)] are defined on multitape Turing machines)

We have seen: for $t(n) \geq n^2$,

$$\mathrm{TIME}[t(n)] \subseteq \mathrm{SPACE}\left[\sqrt{t(n)} \cdot \log t(n)\right]$$

Further improved to: for t(n)≥n, [Williams *STOC 2025*]

$$\mathrm{TIME}[t(n)] \subseteq \mathrm{SPACE}\left[\sqrt{t(n) \cdot \log t(n)}\right]$$

Remark 1.6 (Extension to Oblivious Random-Access Models). *At the present time, we do not know how to extend Theorem 1.1 to arbitrary random-access models of computation. The main issue is that the indegree of the resulting computation graph (defined in Section 3) is so high that the computation graph cannot be stored in* $O(t^{1-\varepsilon})$ *memory. However, we remark that if the pattern of reads and writes of a given random-access machine model is* oblivious, *in the sense that given a step-count* $i = 1, \ldots, t$ *specified in* $O(\log t)$ *bits, the names of the registers being accessed in step* i *can be computed in* $O(\sqrt{t})$ *space, then Theorem 1.1 does apply, with only a* $\mathrm{poly}(\log t)$ *extra space factor. This is because such machines can be simulated by circuits of* $t \cdot \mathrm{poly}(\log t)$ *size, which can in turn be simulated efficiently by multitape Turing machines in* $t \cdot \mathrm{poly}(\log t)$ *time (see Section 4.1).*

FINAL

Final exam

- 13:30, June 9
- Location:6A116
- Fully closed-book

Office hour

- every Monday 4:00-5:00pm, FIT 4-6013

Materials for the final

Lecture	Material covered	Not covered	Pages
1	*none*		--
2	All of finite automata and regular language	DFA Minimizing algorithm	1~120
3	CFG and its pumping lemma	PDA, parsing algorithm	1~50, 89~98
4	All of concepts of TM, different models, Halting Problem	others	1~122
5	Reducibility, recursion theorem, Kolmogorov complexity,	NP-completeness for SAT, Turing degree	1~14, 27~100
6	NL-complete, Savitch's theorem, PSPACE	Immerman's theorem, Gomoku	1~73, 89~127
7	Hierarchy theorems, PH, relativization	Ladner's theorem, alternations	1~14, 45~59, 122~136

Materials for the final

Lecture	Material covered	Not covered	Pages
8	randomized primality test, randomization, BPP $\subseteq$ PH	additive spanner, deterministic primality test	18~96
9	GNI $\in$ IP, IP=PSPACE, decision tree	AM and MA, MIP	1~59, 99~118
10	P/poly, concepts of NC and AC, polynomial method	fast APSP algorithm	1~82
11	*none*		--
12	APSP-hardness, OV, dynamic reachability and diameter	Edit distance, OMv/3SUM-hardness	1~39, 47~62, 92~119
13	understanding “Were-you-last” game	others	1~39
14	understanding basic concepts	$TC^1 \subseteq NL$, tree evaluation, time-space simulation	1~37, 58~63